

day / date:

LECTURE - 13 Tuesday 7/10/25

Union, Intersection & Kleene Star:

$$L_1 \Rightarrow G_1(S_1, \Sigma, V_1, P_1)$$

$$L_2 \Rightarrow G_2(S_2, \Sigma, V_2, P_2)$$

CFL.

$$L_4 = L_1 \cup L_2 : G_4(S_4, \Sigma, V_4, P_4)$$

$$S_4 \rightarrow S_1 / S_2$$

$$V_1 \cap V_2 = \emptyset$$

$$V_4 = V_1 \cup V_2 \cup S_4$$

$$P_4 = P_1 \cup P_2 \cup \{S_4 \rightarrow S_1 / S_2\}$$

one extra

L_1 & L_2 shouldn't have same variable names like: A A with diff def. $\therefore$ after we change variable names there shouldn't be anything common $\therefore \emptyset$.

$$L_1 = \{x \mid x \in \Sigma^* \text{ start}\}$$

$$L_5 = L_1 \cdot L_2 : G_5(S_5, \Sigma, V_5, P_5)$$

$$S_5 \rightarrow S_1 S_2$$

$$V_5 \rightarrow V_1 \cup V_2 \cup S_5$$

$$P_5 \rightarrow P_1 \cup P_2 \cup \{S_5 \rightarrow S_1 S_2\}$$

one extra

$$L_6 = L_1^* : G_6(S_6, \Sigma, V_6, P_6)$$

$$S_6 \rightarrow \Lambda / S_1 S_6$$

$$V_6 \rightarrow V_1 \cup S_6$$

$$P_6 \rightarrow P_1 \cup \{S_6 \rightarrow \Lambda / S_1 S_6\}$$

one extra

$$\Sigma = (a, b)$$

day / date:

Q: $L_3 = \{u \mid u \in \Sigma^* \text{ \& } |u| \text{ is even}\}$
reg. expression: $(aa + ab + ba + bb)^*$

$$S \rightarrow \Lambda / AS$$

$$A \rightarrow aa / bb / ab / ba$$

Q: $L_4 = \{u \mid u \text{ is palindrome}\}$ "abaaba"

$$S \rightarrow \Lambda / a / b$$

$$S \rightarrow aSa / bSb$$

Q: $L_5 = \{u \mid u \in \Sigma^* \text{ \& } a^i b^j \text{ when } i=j\}$

$$S \rightarrow \Lambda$$

$$S \rightarrow aSb$$

$$L_6 = \{u \mid u \in \Sigma^* \text{ \& } a^i b^j\}$$

Q: $L_2 = \{u \mid n(A) = n(B)\}$

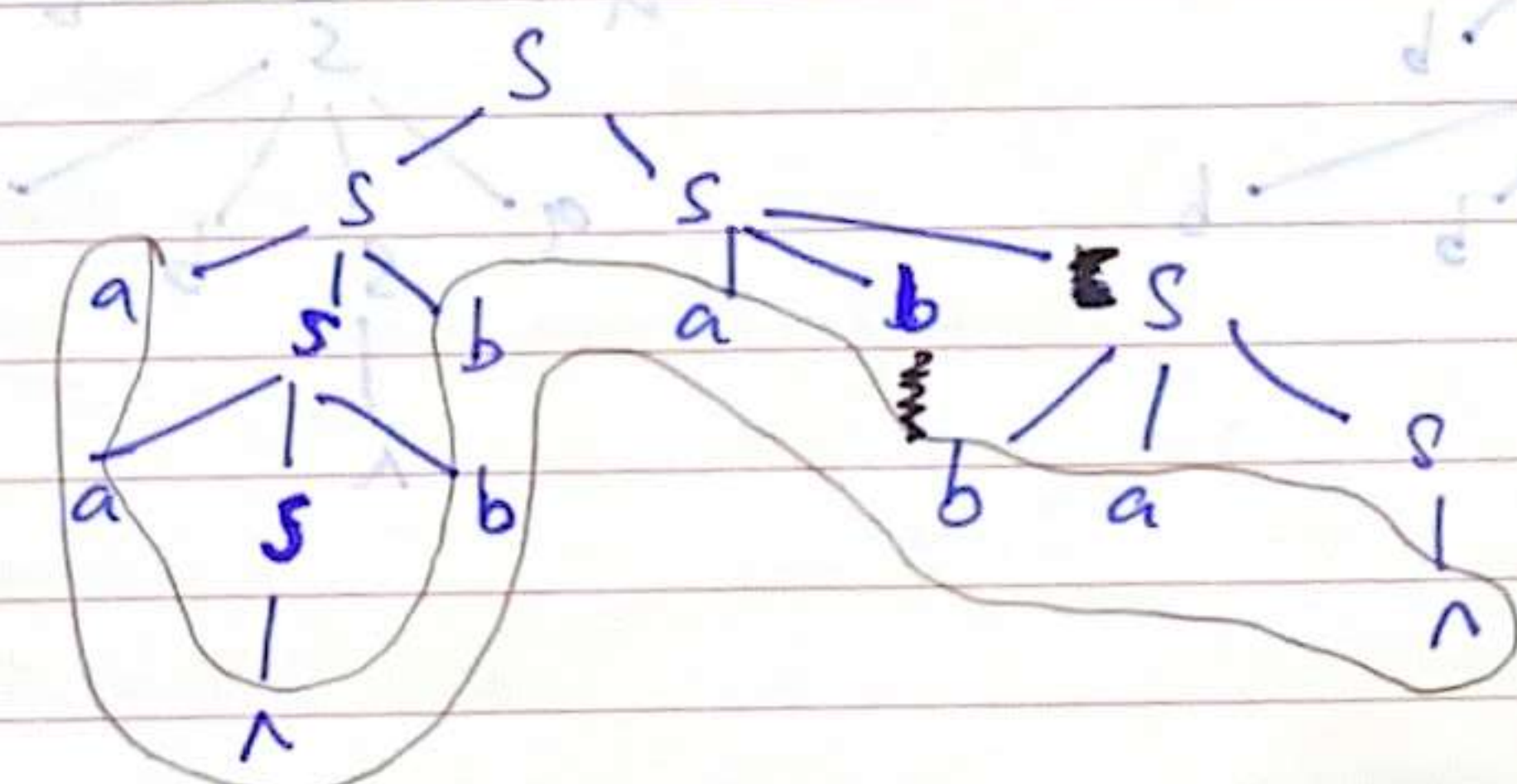
$$S \rightarrow \Lambda$$

$$S \rightarrow aSb / bSa$$

$$S \rightarrow SS$$

(if not this statement; then "abba" not allowed. $\therefore$ "SS" is for mixing of 'a', 'b')

"aabb abba"



Q: $L = \{u \mid u = a^i b^j \text{ where } i > j\}$

$n = a^k a^i b^j$ where $k \geq 1$
 (equal no. of a's & b's (L5 on prev page))
 $B \rightarrow aB/a$ $A \rightarrow aAb/\Lambda$

$\therefore S \rightarrow BA$

Q: $L = \{u \mid u \in \Sigma^* \text{ where } u = a^i b^j \text{ where } i \leq j \leq 2i\}$

i	j
0	0
1	1, 2
2	1, 2, 4
3	3, 4, 5, 6
4	4, 5, 6, 7, 8

Minimum:
 $i = j$

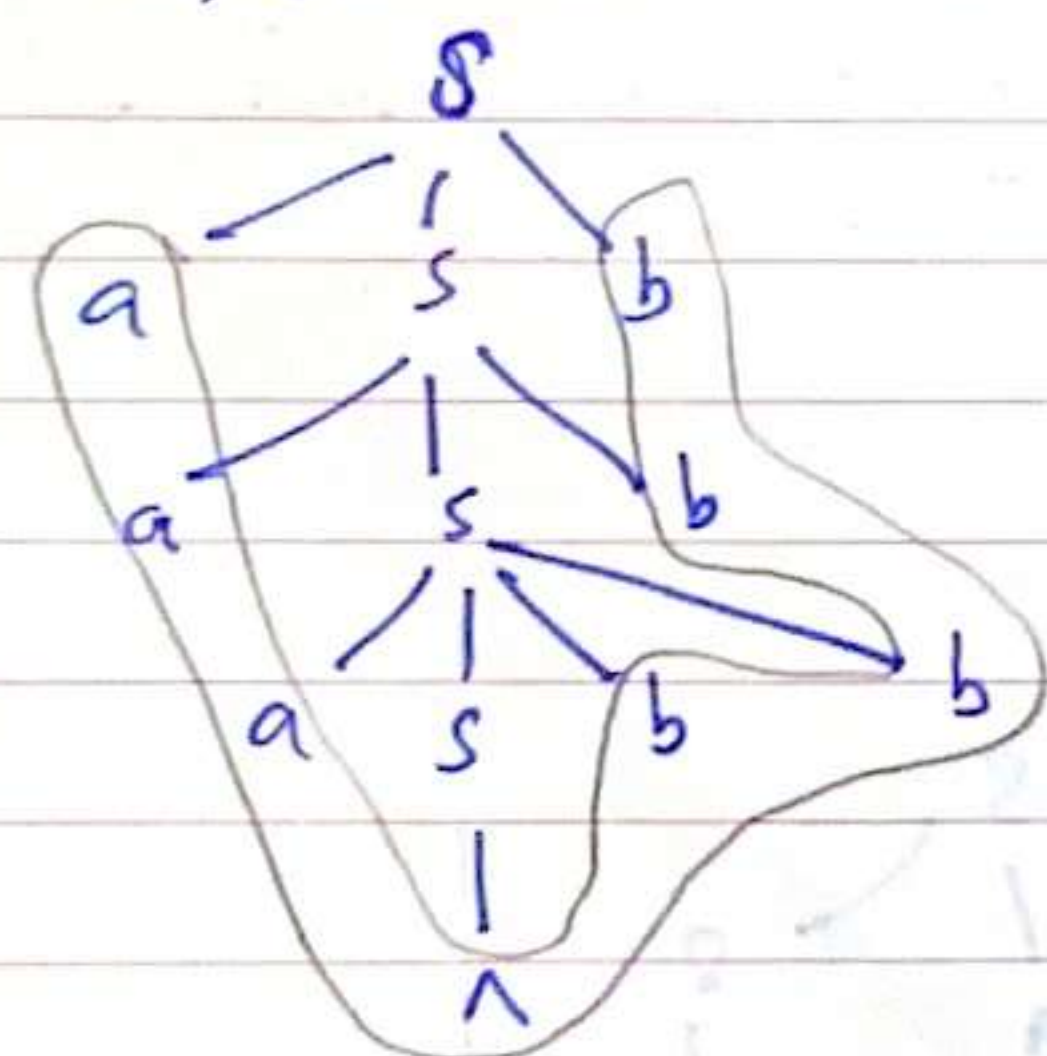
Maximum:
 $j = 2i$

$S \rightarrow aSb/\Lambda$

$S \rightarrow aSbb$

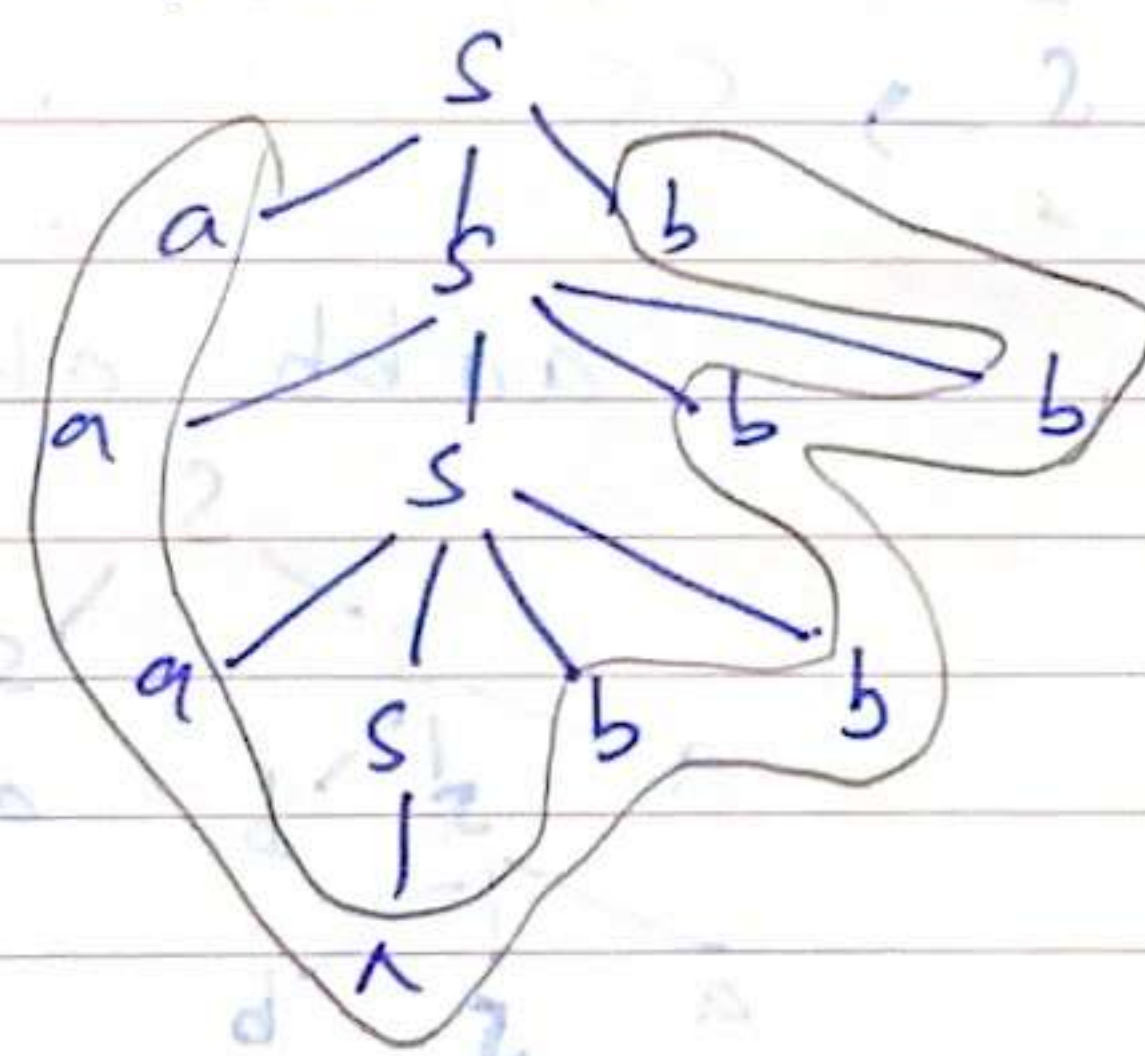
$\therefore S \rightarrow \Lambda$
 $S \rightarrow aSb/asbb$

$i=3, j=4$



aaa bbbb ✓

$i=3, j=5$



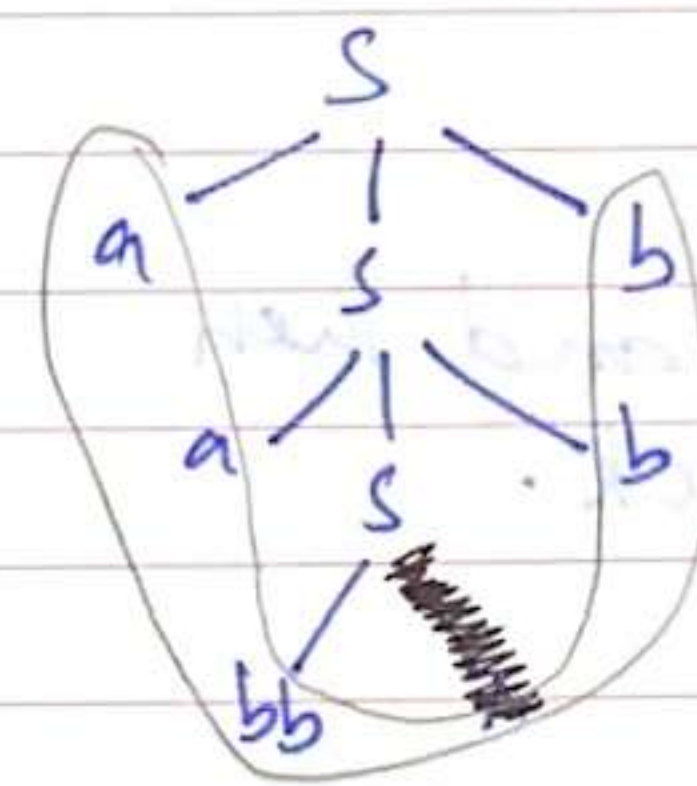
aaa bbbbbb ✓

Q: $\{u \mid u = a^n b^{n+2}\}$

$S \rightarrow aSb/bb$

Two b's added in op condition since power of b was $n+2$.

"a a b b b b"



NOTE: If there is a constant (k) getting added in power of variable like 'n+2'; we'll put that variable 'k' times in termination (i.e.: base case)

LECTURE - 14

Thursday
9/10/25

Language.

① Reg-Expression
 ② FA
 Reg. lang. Non-Reg. lang.

→ DFA
 → NFA
 → NEA NULL

CFL → CFL
 → PDA (FA + Memory) diagram

Dynamic
STACK: using Linked List

already top of stack element
 $a, Z_0 / a Z_0$
 top 2 elements (after pushing)

input:
 about to be added/pushed

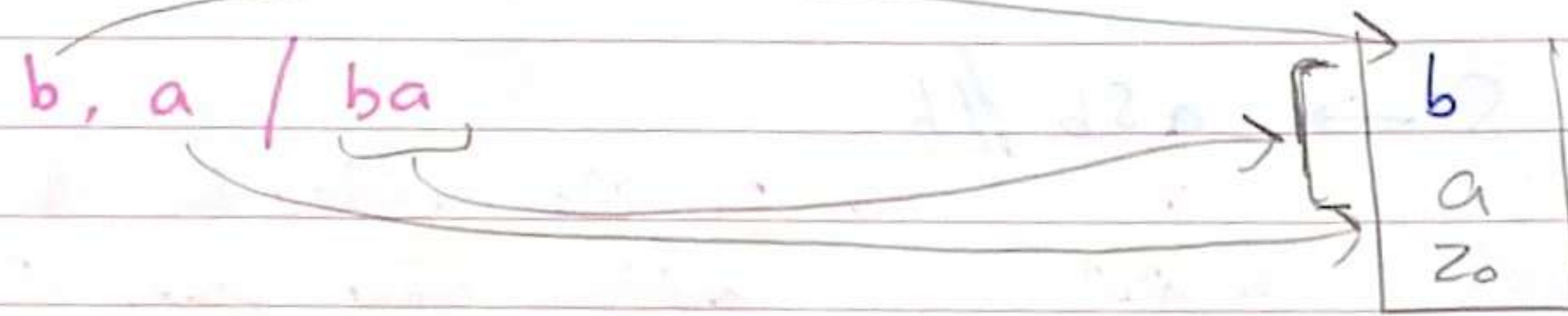
TOS → Z_0

Delimiter:
 if only Z_0 in stack, stack is empty.

day / date:

⇒ PUSH

Inserting 'b' in stack that has 'a' and 'z₀'.



⇒ POP that is coming in input of FA

input, TOS / Λ

• Incoming input is consumed

$a, b / \Lambda$: input: 'a' is consumed and then 'b' is popped out of Stack.

• Incoming input is not consumed.

$\Lambda, b / \Lambda$: b is popped without any input getting consumed.

⇒ Pushing data without consuming input:

Q: Input = ab; add 'aa' whenever there is input 'a'.

$a, z_0 / a z_0$

// 'a' pushed in stack

$\Lambda, a / a a$

// Since first 'a' pushed; ∴ we'll push another

'a' that we're to put ourself.

about to be added. already pushed TOS in stack. This has to be done without consuming next input i.e: 'b'

∴ we used Λ, a not 'ba'.

$b, a / b a$

// 'b' pushed after two 'a's.

Pop	with consuming	$a, b / \Lambda$	
Pop	w/out //	$\Lambda, b / \Lambda$	Example
Push	with //	$a, b / a b$	nothing & do
Push	w/out //	$\Lambda, b / a b$	nothing
Common	Consume but do nothing	$a, b / b a$	$\Lambda, a / a$

* DPDA : Deterministic pushdown automata : only one transition for one set of state & input.

NDPDA : Non-

//

//

day / date:

FA = (Q, Σ , q₀, A, δ)

Stack = (Γ , z₀)

PDA = (Q, Σ , Γ , q₀, A, z₀, δ)

transition func. of PDA: $Q \times \{ \Sigma \cup \Lambda \} \times \Gamma \rightarrow Q \times \Gamma^*$

current state, current input, TOS, next states of PDA after changing Q, Σ and Γ

e.g: single/multiple push or pop, no action etc.

since 'c' is pushed here but is not written in the top two element of stack (after slash) ∴ it means 'c' isn't added in stack. as 'c' is consumed but not added.

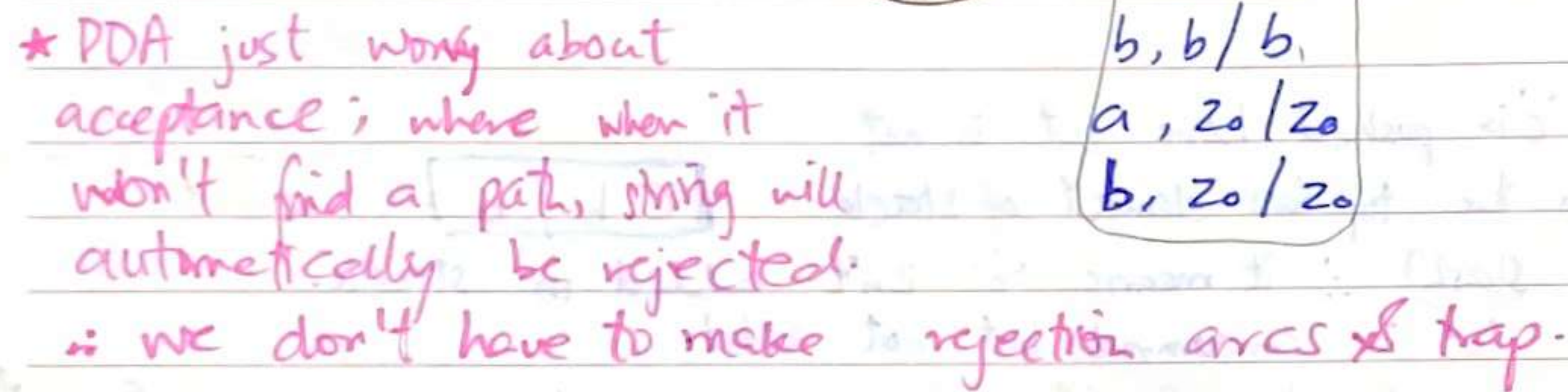
Q: L = { n | n ∈ Σ^* & n = ycy^r where y ∈ {a,b}* }
 δ y^r = reverse of y } . Palindrome
 $a^2 b^3$ & $b^5 a^7$

1 $a, z_0 / a z_0$
 2 $b, a / b a$
 3 $b, b / b b$
 4 $c, b / b$ // no action since 'c' not in top two
 5 $b, b / \Lambda$
 6 $b, b / \Lambda$
 7 $a, a / \Lambda$

LOGIC : cannot be pop operations where current input ≠ TOS else string would be rejected.

e.g: $a, b / \Lambda$ X as current input to be checked is 'a' but the TOS is 'b' so balance of palindrome distributed

day / date:

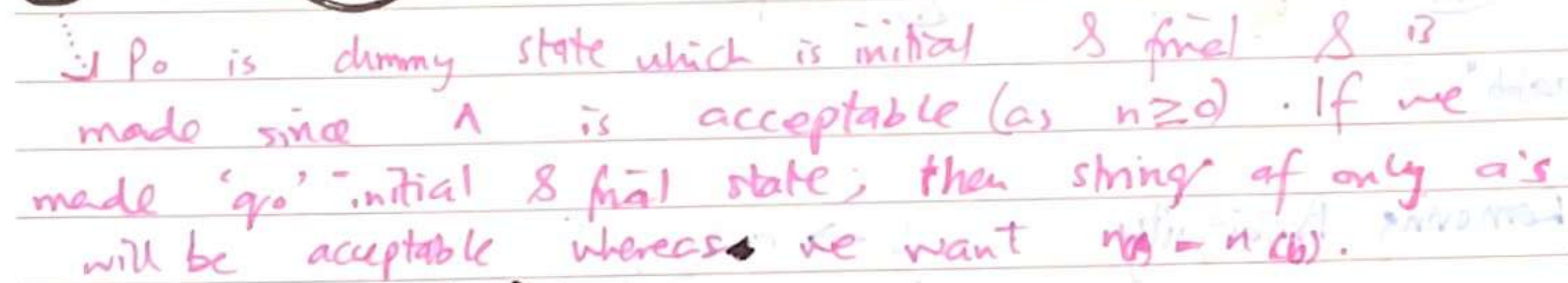


- (State(Q), Input(Σ), TopofStack(Γ))

since final state in answer $\therefore$ $abcba$ accepted.

accepted

day / date;



- ρ_0, ab, z_0
 q_0, b, az_0
 $q_1, \wedge, z_0$
 $q_2, \wedge, z_0$

→ accepted as 'y₂' is final.

LECTURE-15 Tuesday : ONLINE
14/10/25

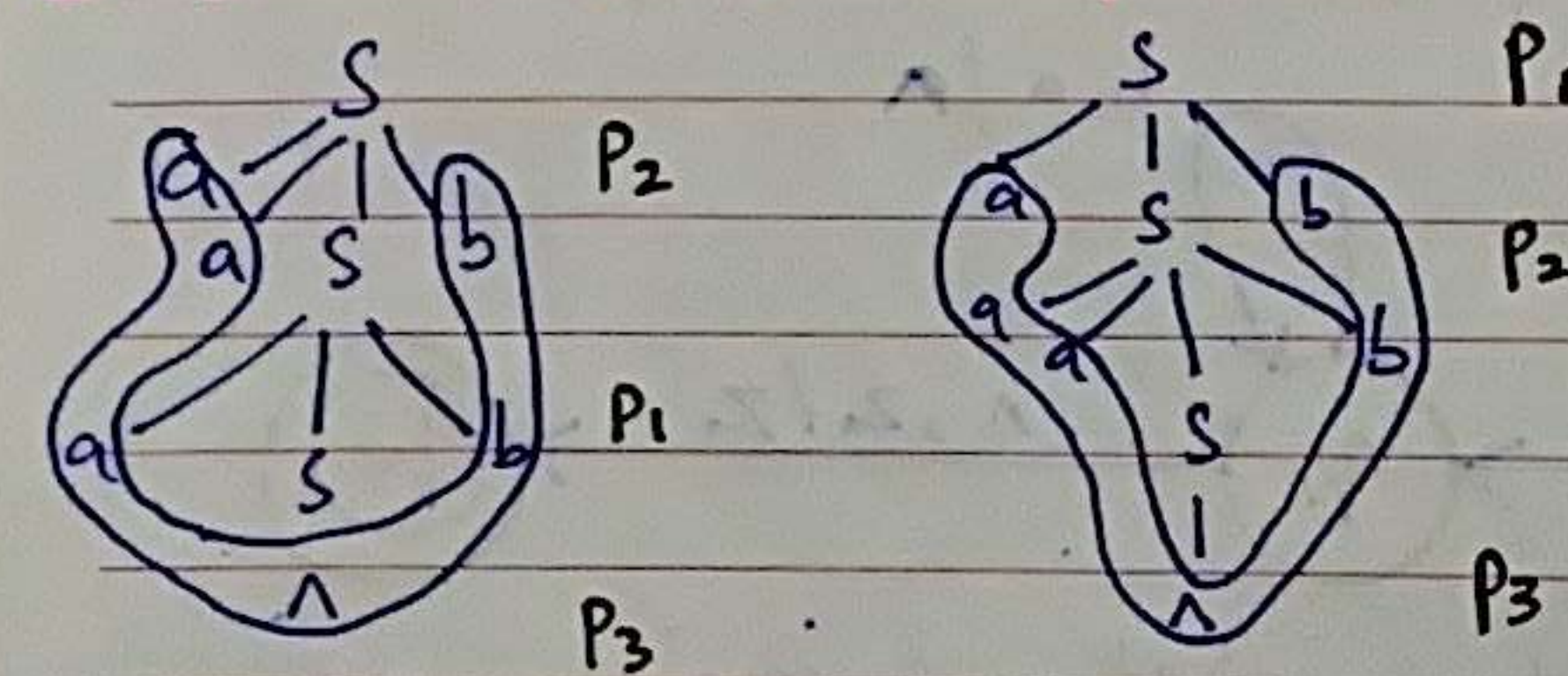
① $S \rightarrow aS / Sa$

$$\begin{array}{c}
 S \\
 \swarrow \searrow \\
 S \quad a \\
 \swarrow \searrow \quad \swarrow \searrow \\
 S \quad a \quad S \quad a \\
 \swarrow \searrow \quad \swarrow \searrow \quad \swarrow \searrow \\
 S \quad a \quad a \quad S \quad a \quad S \quad a \\
 \swarrow \searrow \quad \swarrow \searrow \quad \swarrow \searrow \\
 S \quad a \quad a \quad S \quad a \quad S \quad a \\
 \swarrow \searrow \quad \swarrow \searrow \quad \swarrow \searrow \\
 S \quad a \quad a \quad S \quad a \quad S \quad a
 \end{array}$$

- Removing : Sa/n OR (aS/n)
Ambiguity

day / date:

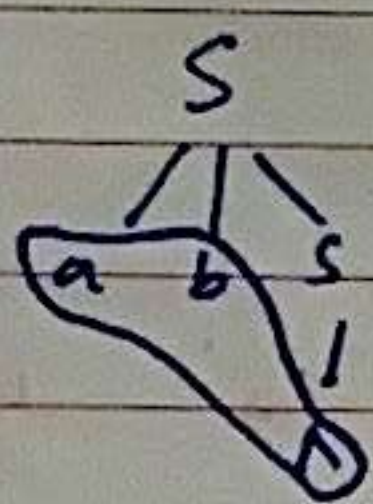
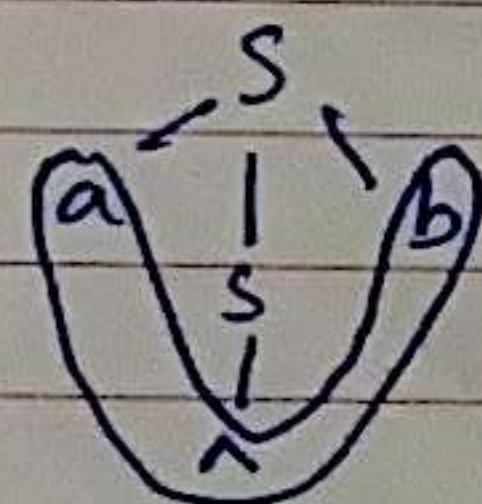
② $S \rightarrow aSb / aaSb / \Lambda$



"aaabbb"

• Removing Ambiguity : $S \rightarrow aSb / T$
 $T \rightarrow aaTb / \Lambda$

③ $S \rightarrow aSb / abS / \Lambda$: Ambiguous on "ab".



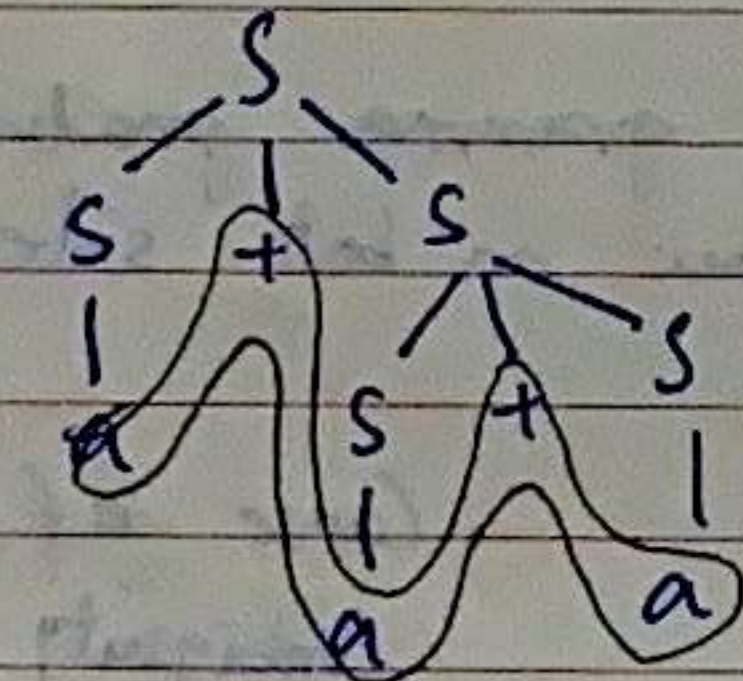
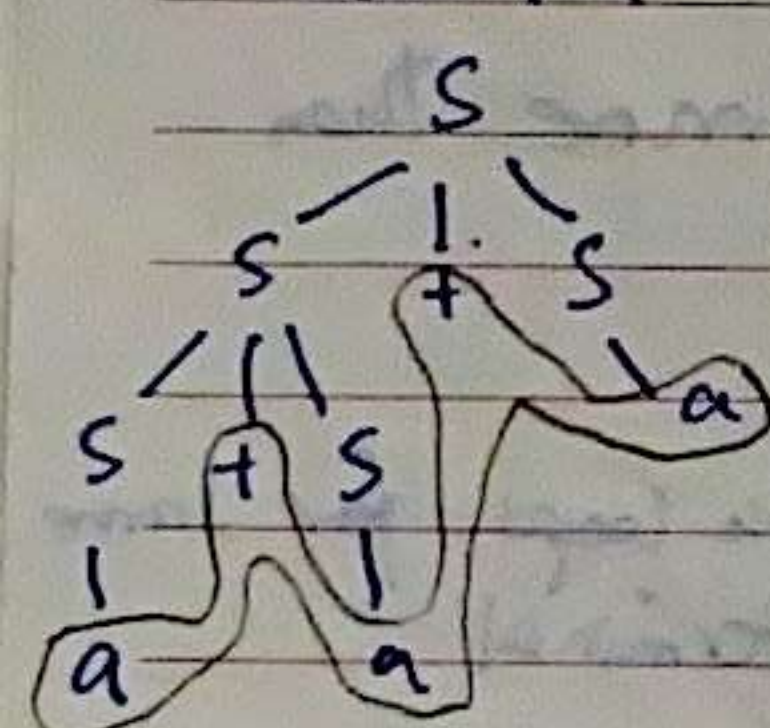
Removing Ambiguity : $S \rightarrow \Lambda / A$
 $A \rightarrow aAb / abA / ab$

④ $S \rightarrow S+S / a$: $\Sigma = \{a, +\}$

replace 'S' with a terminal

• Remove Ambiguity : $S \rightarrow a+S / a$

• Cause of Ambiguity : 'S' on both sides of '+'



non-deterministic : when we need to guess stuff like in this case center of string.

day / date:

PDA
 NPDA

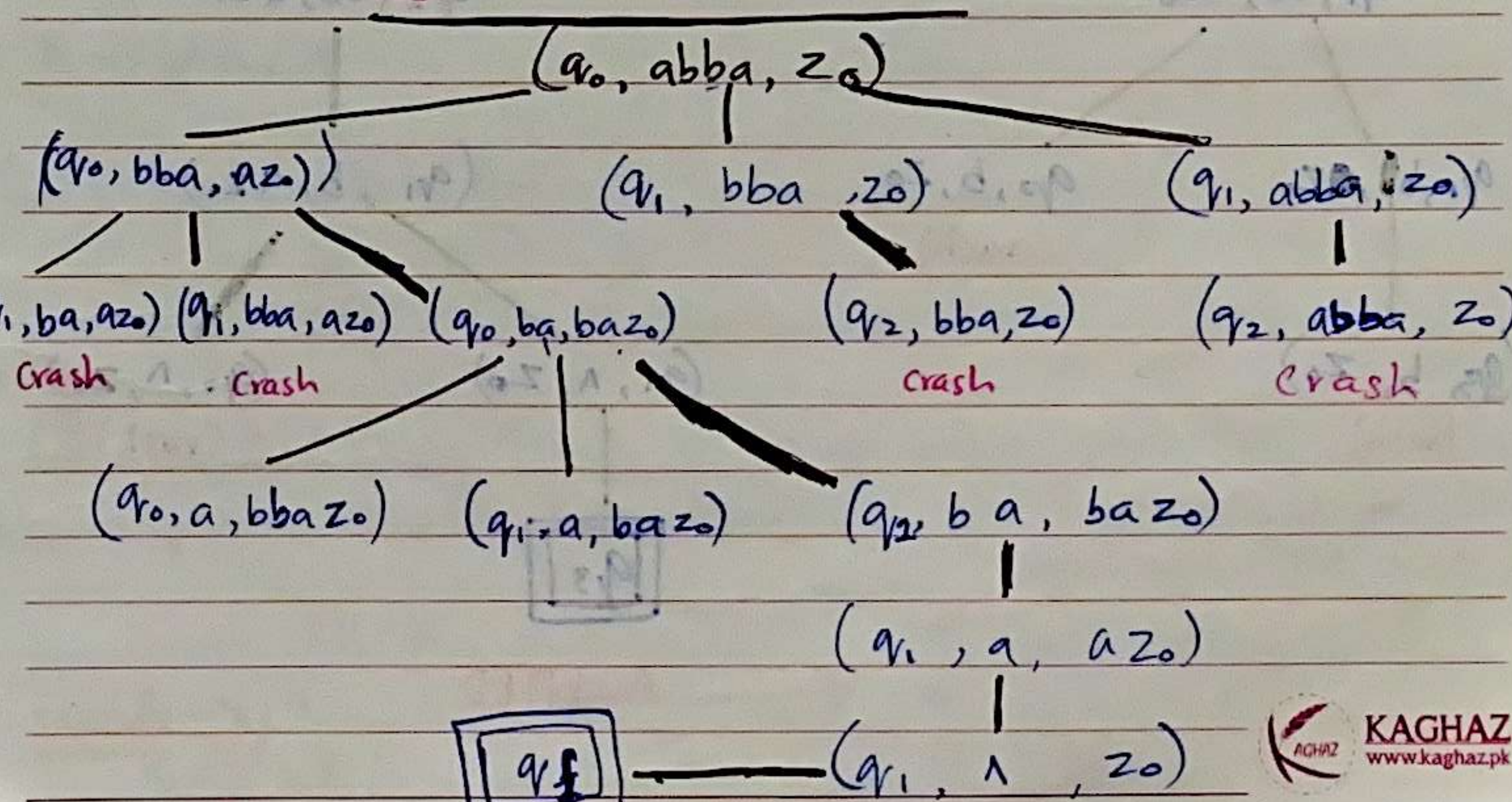
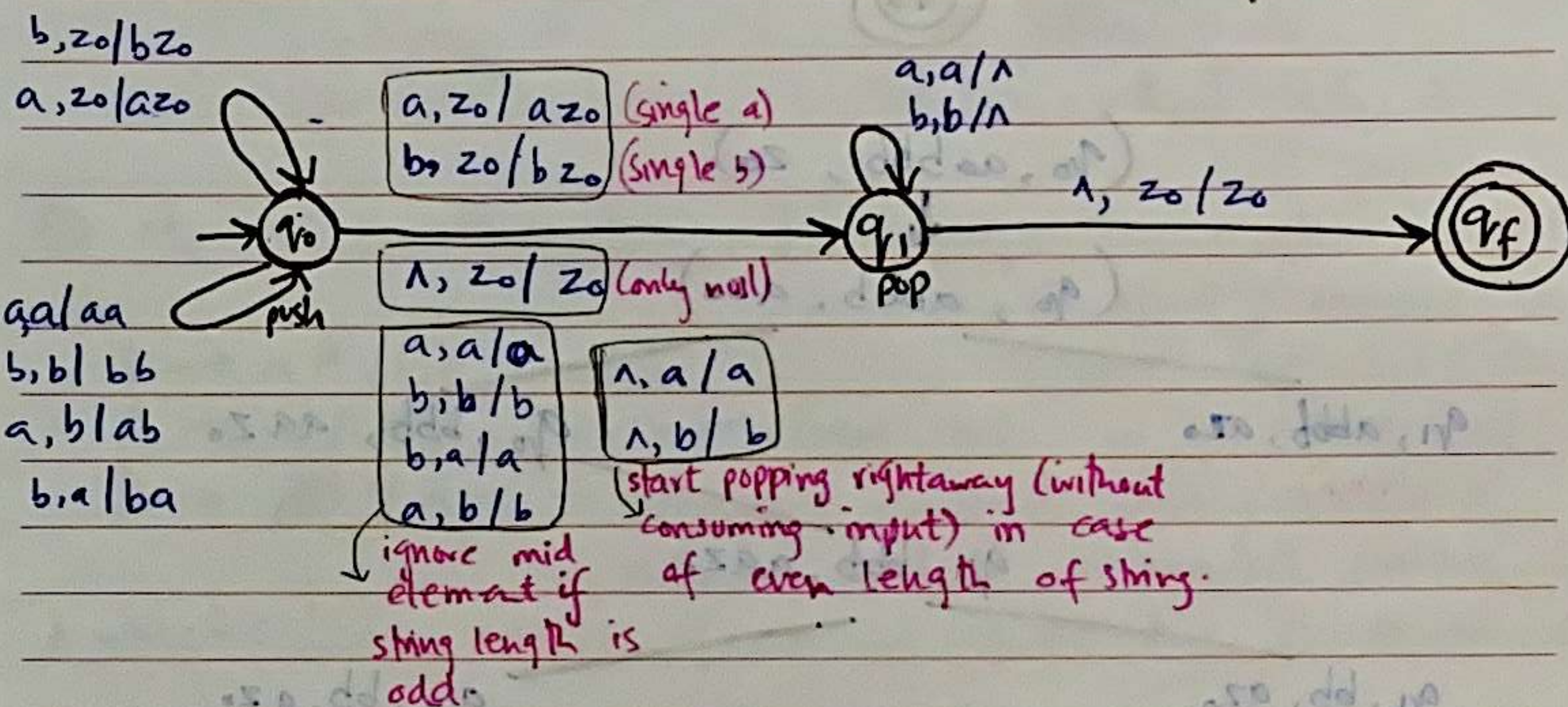
LECTURE-16

Thursday
 16/10/25

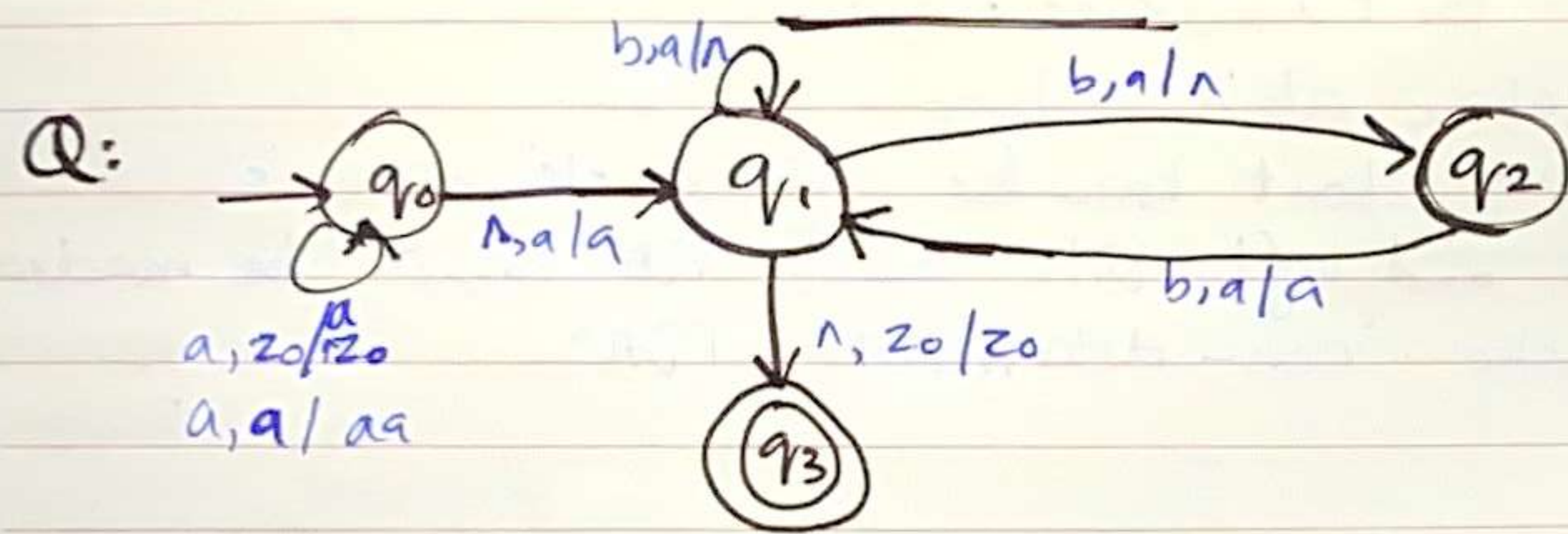
NPDA for Palindrome: (without 'c' in middle)

$a, babba, aba, ab\Lambda, baab, aa, bb$

• Since we don't know the middle element 'c' (jiski right and left side equal): PDA cannot be made. we'll make "non-deterministic PDA" (more than 1 arcs outgoing).



★ The tree on prev. page was to show if "abba" gets accepted or rejected.
 ↓ It got accepted since q_2 (final state) in answer.



$(q_0, aabbb, z_0)$

$(q_0, abbb, az_0)$

$q_1, abbb, az_0$
(crash)

$q_0, bbb, aa z_0$

$q_1, bbb, aa z_0$

q_1, bb, az_0

q_2, bb, az_0

q_1, b, az_0

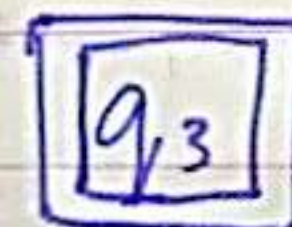
q_2, b, z_0
(crash)

(q_1, b, az_0)

(q_3, b, z_0)
(crash)

$(q_1, λ, z_0)$

$(q_2, λ, z_0)$
(Crash)



ACCEPTED

as we've reached final state but input buffer isn't empty (it has 'b')

LECTURE - 17

Tuesday
day / date: 21/10/25.



CNF: Either

- exactly 2 variable by a variable (like: $S \rightarrow AB$)
- OR
- Variable $\rightarrow$ single terminal (like $S \rightarrow a$)

① No Useless production:

Ex: $S \rightarrow S/a$

useless production of 'S'

Solution: $S \rightarrow a$

as $S \rightarrow S$ isn't producing any terminal $\therefore$ useless.

Ex: $S \rightarrow A/B$

$A \rightarrow aAa$

$B \rightarrow bBb$

Infinite loop on A/B since 'A' never ends.

Solution: $S \rightarrow B/A$

② No NULL Production: If language has λ , then it could only exist with starting variable (i.e. S)

Ex:

$S \rightarrow aA/Bb$

$A \rightarrow cAc/\lambda$

$B \rightarrow dB/\lambda$

remove these nulls in a way that language doesnot changes. as ' λ ' is with A & B which are starting variable

$A \rightarrow cAc/cc$ instead of λ we can use this 'cc' to terminate

$B \rightarrow dB/d$

$S \rightarrow aA/a/Bb/b$

edge case of single a or b in string.

③ No Unit Production: If we've single variable, we'll replace it with the definition of that single variable

$S \rightarrow AB/A$

$A \rightarrow a$

$B \rightarrow bB/b$

unit production

$S \rightarrow AB/a$

$A \rightarrow a$

$B \rightarrow bB/b$

solution

Q: $S \rightarrow ABAC$

$A \rightarrow aAa/\lambda$

$B \rightarrow bB/\lambda$

$C \rightarrow dCa/\lambda$

Remove ~~useless~~ ^① ~~NULL~~ ^② productions.

language accepts λ $\therefore$ we'll add λ in 'S' starting production.

$S \rightarrow ABAC / BAC / ABC / BC / AAC / ABA / AA/\lambda / BA / AB/AC / A/B/C$

$A \rightarrow aAa / aa$

$B \rightarrow \text{~~baa~~} bB/b$

$C \rightarrow dCa/da$

Q: $S \rightarrow ACD$ $E = \{a, b\}$ variables = $\{S, A, C, D\}$

$A \rightarrow aAb/\lambda$

$C \rightarrow aC/a/C$

$D \rightarrow aDa/bDb/\lambda$

① Remove useless produc.

$S \rightarrow ACD$

$A \rightarrow aAb/\lambda$

$C \rightarrow aC/a$

$D \rightarrow aDa/bDb/\lambda$

② Remove NULL production

$S \rightarrow ACD / AC / CD / C$

$A \rightarrow aAb / ab$

$C \rightarrow aC / a$

$D \rightarrow aDa / bDb / aa / bb$

③ Remove Unit Production

$S \rightarrow ACD / AC / CD / aC / a$

$A \rightarrow aAb / ab$

$C \rightarrow aC / a$

$D \rightarrow aDa / bDb / aa / bb$

Remove NULL productions.

Q: $S \rightarrow ABC$

$A \rightarrow aA/\lambda$

$B \rightarrow BB A/B$

$S \rightarrow ABC / Bc$

$A \rightarrow aA/a$

$B \rightarrow BbA / Bb/b$

④ Make right side of arrow of variable ^{all} a combo of either ^{just} all variables (A/B/C) or single variable/terminal (a). (Not like aA or bBa)

$S \rightarrow ACD / AC / CD / XC / a$

$X \rightarrow a$

$Y \rightarrow b$

$A \rightarrow XAY / XY$

$C \rightarrow XC / a$

$D \rightarrow XD / YDY / XX / YY$

replace "aC" which was a combo of single terminal (i.e: a) and a variable (C) which isn't acceptable.

⑤ Make the right side of arrow which has either 2 variables or a single terminal:

we just have to slice the expression with more than 2 variables into a new expression with 2 variables exactly by introducing a new variable

$S \rightarrow AT / AC / CD / XC / a$

$T \rightarrow CD$ ^{new}

$X \rightarrow a$

$Y \rightarrow b$

$A \rightarrow XW / XY$

$W \rightarrow AY$ ^{new}

$C \rightarrow XC / a$

$D \rightarrow XZ / YQ / XX / YY$

$Z \rightarrow DX$ ^{new}

$Q \rightarrow DY$ ^{new}

LECTURE - 18

Thursday
23/10/25

□ Parser: To check whether a string is part of language or not.

- To do this parser would traverse the tree it made either via Top to down OR Bottom to Top.

Bottom UP Parser: method → CYK (Cooke Younger Kasni)

- To apply CYK method for bottom up parser; grammar should be in CNF form (2 variables / single terminal)

$S \rightarrow XY$
 $X \rightarrow XA/a/b$
 $Y \rightarrow AY/a$
 $A \rightarrow a$

already in CNF

String: "babaa"

with these direct rules of grammar we can produce this 'a'

				a	
				X, Y, A	
			a	X, Y, A	S, X, Y
		b	X	S, X	S, X
	a	X, Y, A	φ	φ	φ
b	X	S, X	φ	φ	φ

Step 1: 1st diagonal

cartesian product of immediate left & immediate top cell.

Step 2: 2nd diagonal

$$(1, 2) = (1, 1) \times (2, 1) = \{X\} \times \{X, Y, A\}$$

$$= \{XX, XY, XA\}$$

- From grammar we'll see ^{how} each element of cell will be produced.

$$= \{\phi \cup S \cup X\} \xrightarrow{X \rightarrow XA}$$

since nothing can produce XX

$$S \rightarrow XY$$

$$\therefore (1, 2) = S, X$$

$$(2, 2) = (2, 1) \times (3, 1)$$

$$= \{X, Y, A\} \times \{X\}$$

$$= \{XX, YX, AX\}$$

can't be XY since order matters

$$= \{\phi \cup \phi, \phi\}$$

$$(2, 2) = \phi$$

Step 3: 3rd diagonal → cartesian product with

[left most and immediate top] U [next left (right of left) and next top] U [next left and next top] ...

$$(1, 3) = \{(1, 1) \times (2, 2)\} \cup \{(1, 2) \times (3, 1)\}$$

left most immediate top next left next top

$$\{X\} \times \{\phi\} \cup \{S, X\} \times \{X\}$$

$$\phi \cup \{SX, XX\}$$

$$\phi \cup \{\phi, \phi\}$$

$$(1, 3) = \phi$$

day / date:

$$\begin{aligned}
 \cdot (2,3) &= [(2,1) \times (3,2)] \cup [\{2,2\} \times \{4,1\}] \\
 &= [\{x,y,A\} \times \{s,x\}] \cup [\emptyset \times \{x,y,A\}] \\
 &= \{xs, ys, AS, xx, yx, Ax\} \cup \{\emptyset\} \\
 &= \emptyset \cup \emptyset \\
 (2,3) &= \emptyset
 \end{aligned}$$

Step 4: 4th diagonal: Same Rule as step 3.

→ (left most $\times$ immediate top) $\cup$ (next left $\times$ next top)
 $\cup$ (next left $\times$ next top)

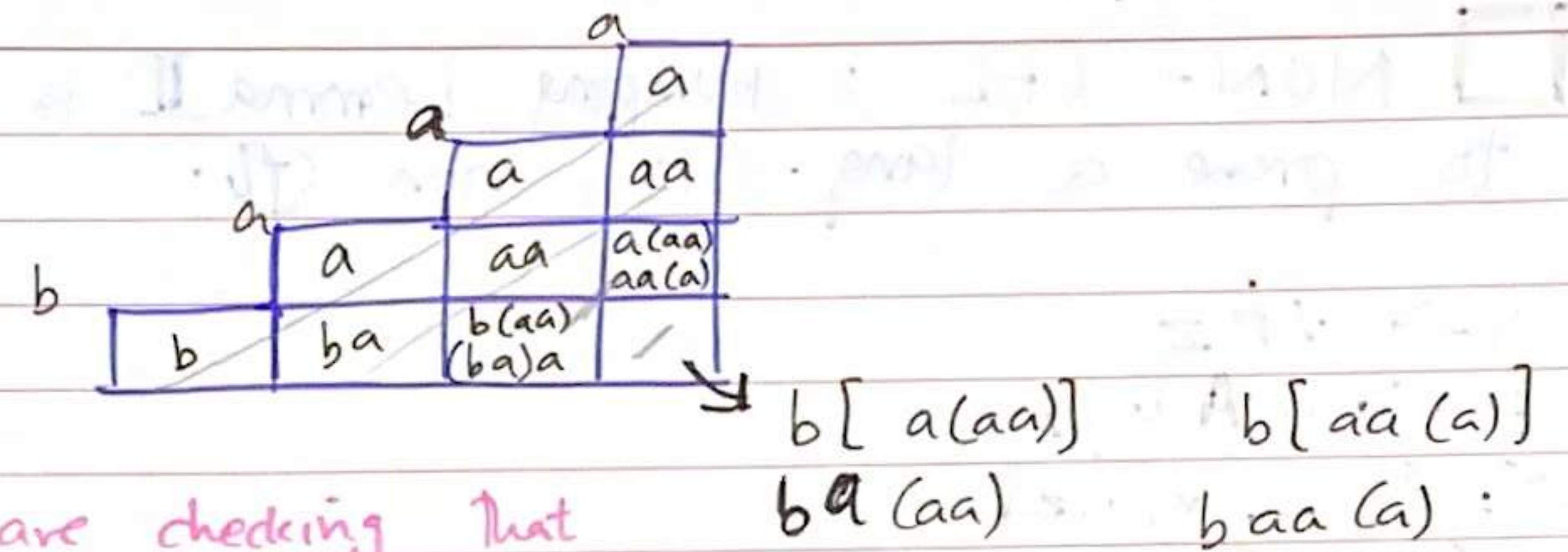
$$\begin{aligned}
 (1,4) &= [(1,1) \times (2,3)] \cup [(1,2) \times (3,2)] \cup [(1,3) \times (4,1)] \\
 (1,4) &= \emptyset
 \end{aligned}$$

Step 5: 5th diagonal (Bottom right cell):
 same rule as ④ & ③.

ANSWER: If after table completion, the bottom right cell (5th diagonal) ~~does not have~~ starting variable (i.e.: S); String would be rejected else accepted.

Accepted String: "baaa"

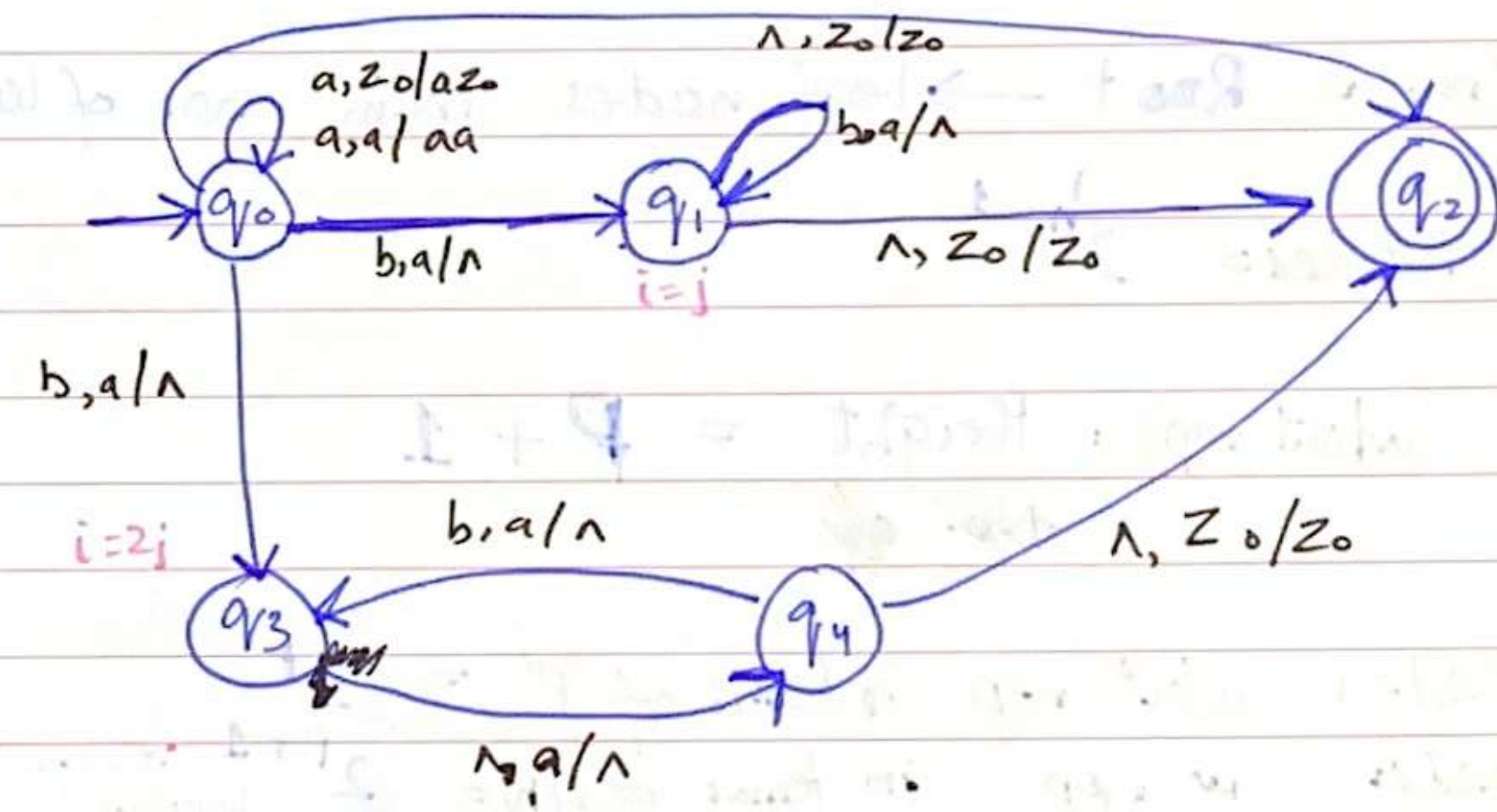
day / date:



Here we are checking that in each cell, which substrings of the original string (i.e.: "baaa") is being produced.

LECTURE-19 Thursday 30/10/25.

Q: Develop PDA & CFG
 $L = a^i b^j$ where $i=j$ or $i=2j$



⇒ Making PDA of a reg. lang: A regular lang. will have DFA ∴ to convert it into PDA we'll put same instruction on every arrow as i.e.: $a, z_0/z_0$ $b, z_0/z_0$

means input is consumed but nothing is pushed or popped as we don't need stack (memory) in reg. lang.

day / date:

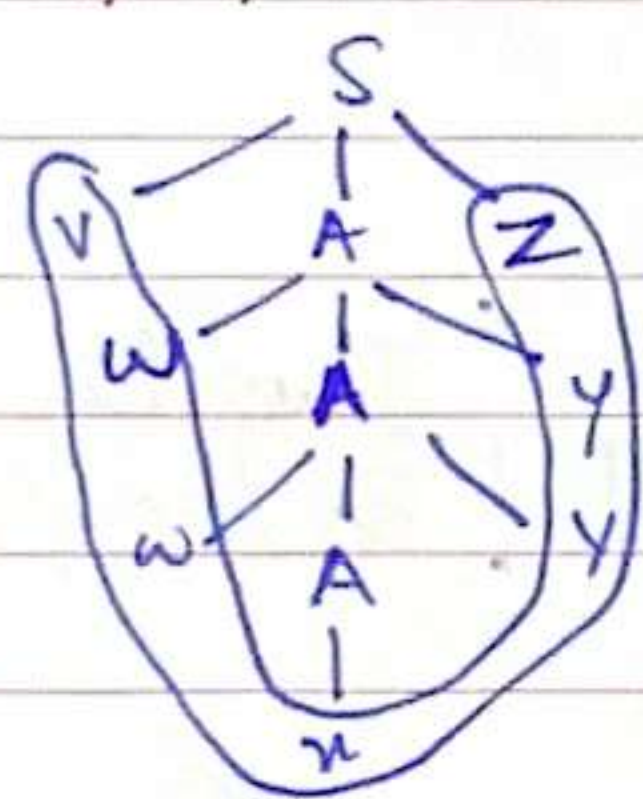
□ NON-CFL : Pumping Lemma II is used to prove a lang. is non CFL.

$$S \rightarrow \sqrt{A_z}$$

$$A \rightarrow \omega A y/x$$

$$\Sigma = \{v, w, x, y, z\}$$

$$V = \{A, S\}$$



$$= V \omega^2 n_y^2 z$$

As 'A' is recursively being called,
So the string which looks like vw^2xy^2z
with powers of 'w' and 'y' being
anything would be acceptable

(h) Height of Tree = Root $\rightarrow$ leaf nodes min no. of levels

No. of leaf Nodes = 2^{h-1}

If 'P' variables (w/out rep): $\text{Height} = P + 1$

No. of leaf Nodes w/out rep in terms of 'P' = 2^P
 No. of leaf Nodes w rep in terms of $P(n)$ = 2^{P+1} recursion involved somewhere.

String $U = v \overbrace{w \ x \ y}^{\text{swap condition}} z$ $|U| \geq n$

before recursion recursion after recursion

① $|w_y| \geq 1$

left recursion: $y = \Lambda$

right " : $w = \Lambda$

middle " : $y \neq \perp, w \neq \perp$

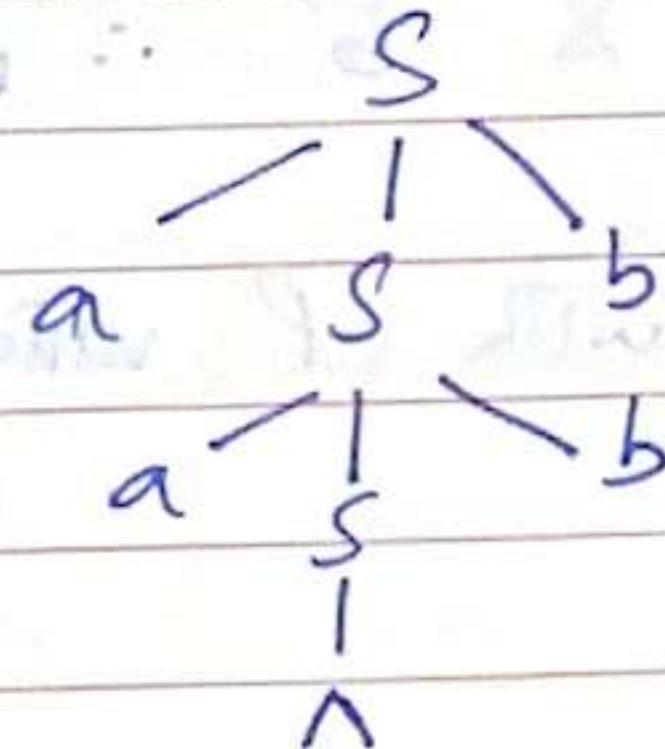
(2) $|wxy| \leq n$

$$vw^iuz \in L \text{ for all } i \geq 0$$

day / date:

middle recursion

$$S \rightarrow aSb / \Lambda$$



$$w = a$$

$$y = b$$

$y = b$
 $x = a$ (Swap condition)

$v, z = \Lambda$ (as no part before/after recursion).

① $|wy| > 0$ (explanation) : means recursion is definitely involved as the length of string has exceeded no. of variables.

② $|way| \leq n$

0 from ① } $0 + n = n$
n from ② } .

④ $0 \leq |x| \leq n-1$

$$1 \leq |w(y)| \leq n$$

recursion
stopped
using Δ

recursion stopped using a string like

$$\left. \begin{array}{l} n-1 \text{ from (1)} \\ 1 \text{ from (2)} \end{array} \right\} (n-1) + 1 = n$$

Method: Assume lang is CFL which will have CFG and that means it should hold Pumping lemma **III** property. But if we prove, any string not ~~fulfilling~~ ~~fulfilling~~ lemma property; that shows the lang is non-CFL.

- for that we'll assume $n = 2^{p+1}$ means recursion defo. involved.

Q: $L = a^i b^i c^i$ where $i \geq 0$: Since we can't keep track of 'i' with 'a' & 'b' $\therefore$ not CFL.

- Let 'L' be CFL $\Rightarrow$ CFG exist with P variables
 $n = 2^{P+1}$

$|u| \geq n$ & $u \in L$ $\xrightarrow{\text{string length greater than 'n' and recursion defo.}}$

$U = a^n b^n c^n \rightarrow \text{Instance}$
 $|U| = n + n + n \therefore \boxed{3n > n}$

$U = vwxyz$

Case 1: $v = \Lambda$, $wxy = a^n$, $z = b^n c^n$ [$|wxy| = n$].

$\neq wxy = a^n$

$x = \Lambda$, $w = a^{n/2}$, $y = a^{n/2}$ $|wy| > 0$ & $|wxy| \leq n$

$vw^i xy^i z = (\Lambda) (a^{n/2})^i (\Lambda) (a^{n/2})^i (b^n c^n) \dots \text{for all } i \geq 0 \dots$

i=1 $\rightarrow a^{n/2} a^{n/2} b^n c^n$
 $\hookrightarrow$ this belongs to 'L' $\therefore$ choose diff 'i'.

i=2
 $vw^2 xy^2 z = (a^{n/2})^2 (a^{n/2})^2 (b^n) (c^n)$

$= a^{2n} b^n c^n \notin L$

(as no. of 'a' greater than 'b' and 'c').

- $x = \Lambda$, $w = a^{n-1}$, $y = a$ [$|wxy| = n-1$]
- $x = a^{n-1}$, $w = a$, $y = \Lambda$ (or $x = a^{n-1}$, $w = \Lambda$, $y = a$) [$|wxy| = 1$]

Case 2: $v = \Lambda$, $wxy = a^k$, $z = a^{n-k} b^n c^n$

Case 3: $v = a^n$, $wxy = b^n$, $z = c^n$

Case 4: $v = a^n b^n$, $wxy = c^n$, $z = \Lambda$

~~Case~~ * Only some cases need to be mentioned and subcases of single case need to be solved.



⇒ After Mid#2

day / date:

LECTURE-20

Thursday
6/11/25

~~Q: L = {u | u = ss, s ∈ Σ*~~ } Pumping
Σ = (a, b). Prove 'L' is non CFL. Pumping Lemma II
Halving the even string will produce 2 similar parts/substring
Let L be CFL with P-variables

$$n = 2^{k+1}$$

Instance → $u = a^n b^n a^n b^n$
 $|u| = n+n+n+n \rightarrow 4n > n$ ✓

$$u = vwxyz$$

Case 1: $v = a^n, wxy = b^n, z = a^n b^n$.

• $v = a^n, w = \Lambda, x = \Lambda, y = b^n, z = a^n b^n$.

$$vw^i xy^i z = (a^n) (\Lambda)^i (\Lambda) (b^n)^i (a^n b^n)$$

At i=0
 $a^n (b^n)^0 a^n b^n = a^n a^n b^n \notin L$
(as right & left side not equal)

• $v = a^n, w = b^n, x = \Lambda, y = \Lambda, z = a^n b^n$.

$$vw^i xy^i z = a^n (b^n)^i (\Lambda) (\Lambda)^i a^n b^n$$

At i=0; $a^n (b^n)^0 a^n b^n = a^n a^n b^n \notin L$.

day / date:

• $v = a^n, w = b^k, x = \Lambda, y = b^{n-k}, z = a^n b^n$
 $vw^i xy^i z = a^n (b^k)^i (\Lambda) (b^{n-k})^i a^n b^n$

At i=0 → $a^n a^n b^n \notin L$

Case 2: $v = \Lambda, wxy = a^n, z = b^n a^n b^n$

Case 3: $v = a^n b^n, wxy = a^n, z = b^n$

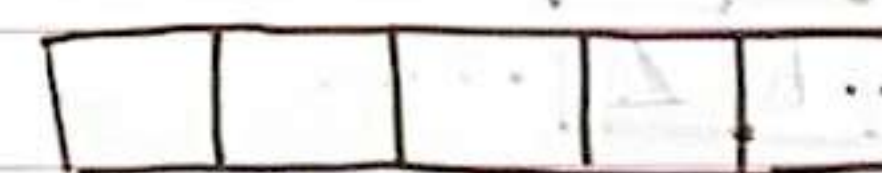
Case 4: $v = a^n b^n a^n, wxy = b^n, z = \Lambda$

Case 5: $v = a^n b^{n-k}, wxy = b^k, z = a^n b^n$

NOTE: We've to list multiple case like (1-5) but we've to solve only 3-4 sub-cases like we did in case 1.

TURING MACHINE: In regular language; we didn't have memory

- In CFL; we've memory but it's limited.
 - This problem of limited memory is solved by Non-CFL.
- Machine of Non-CFL (like: PDA for CFL)



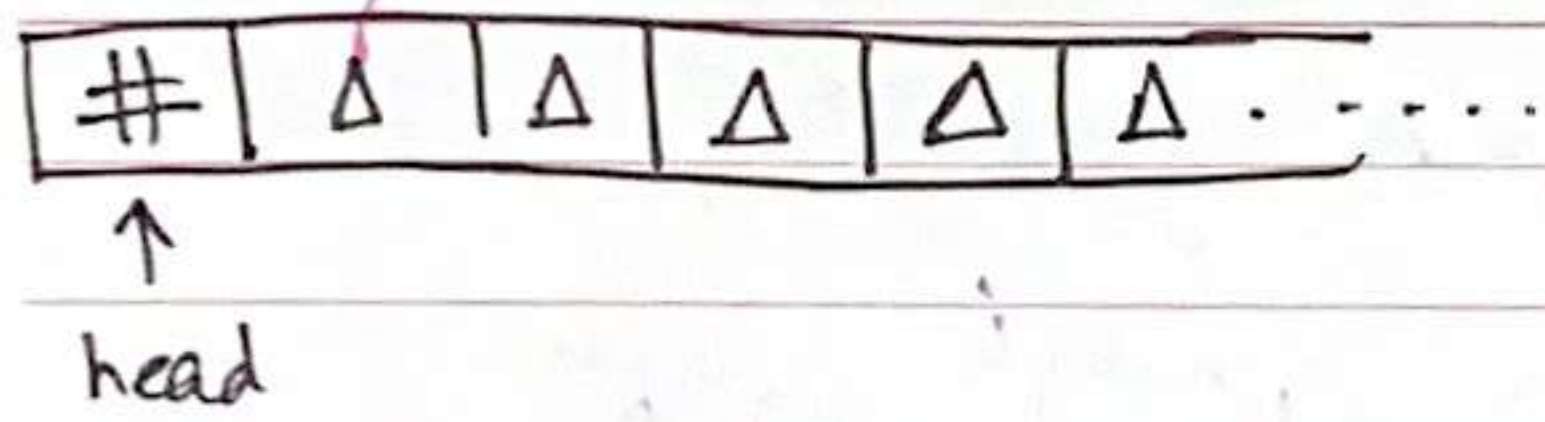
Head

1D array / Tape

- In this array; we have sequential ~~access~~ not random access. Means our head will move to go to index and we can't direct go from $arr[1] \rightarrow arr[5]$.
- Head can move towards left, right or stationary.

delta: means cell is empty.

day / date:



- The first element of tape is always '#' as delimiter to tell tape is empty & we cannot move "left" if we've # in above case.
- # cannot be placed anywhere else in the tape.

Δ input Δ: input ~~string~~ is always placed b/w two deltas

e.g.

#	Δ	a	b	b	Δ
---	---	---	---	---	---

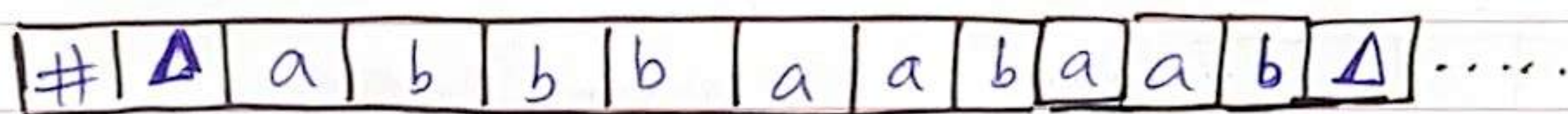
 ΔabbΔ

Q: $L = \{u \mid u \in \Sigma^* \text{ \& \& } u \text{ ends with } ab\}$.

- A halt is produced when a decision is made and in Turing machine; a decision could be made only once.

- ha: halt accept (string accepted)
- hr: halt reject ("rejected")

★ Implicit rejection
Like PDA; we don't need to worry about trap/reject cases & we don't specifically need to draw them. If a path is not found; machine crashes itself.

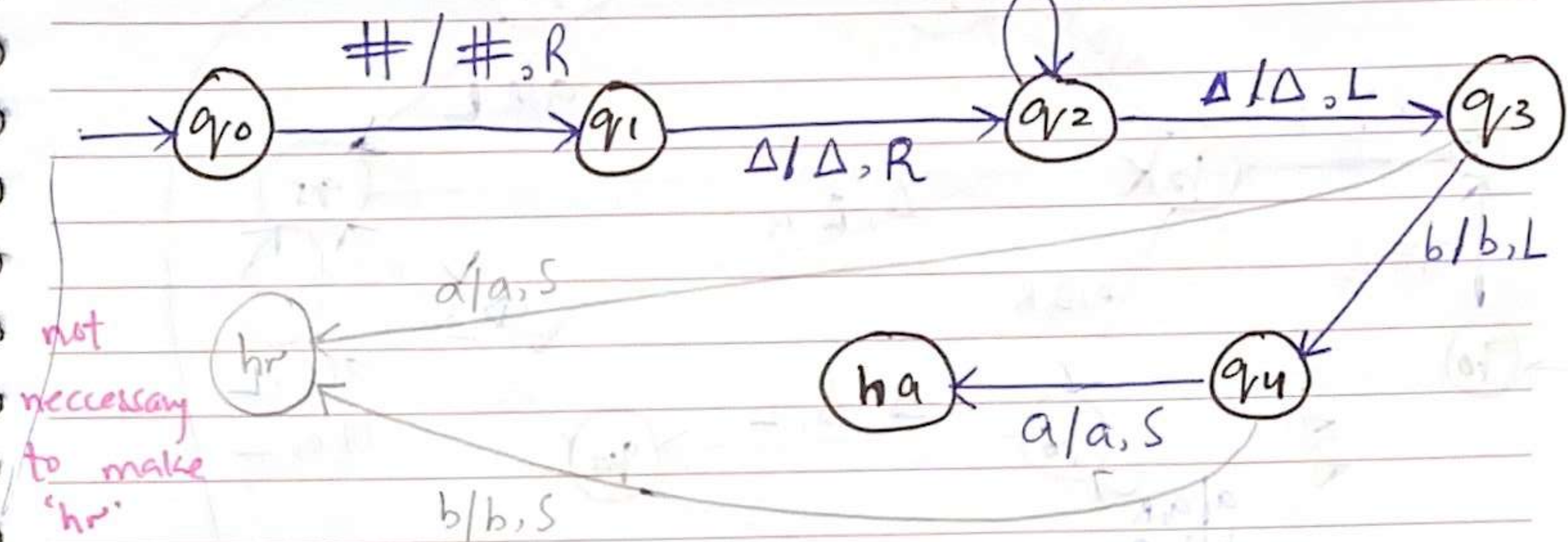


/ #, R

current character

direction (L, R, S).

day / date:
a/a, R
b/b, R



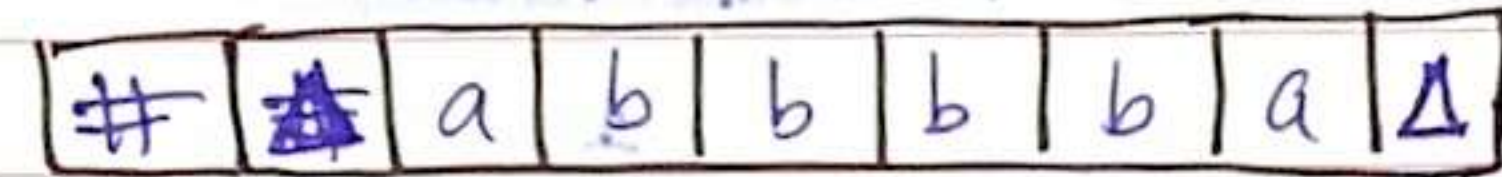
LECTURE-21

Saturday
8/11/25

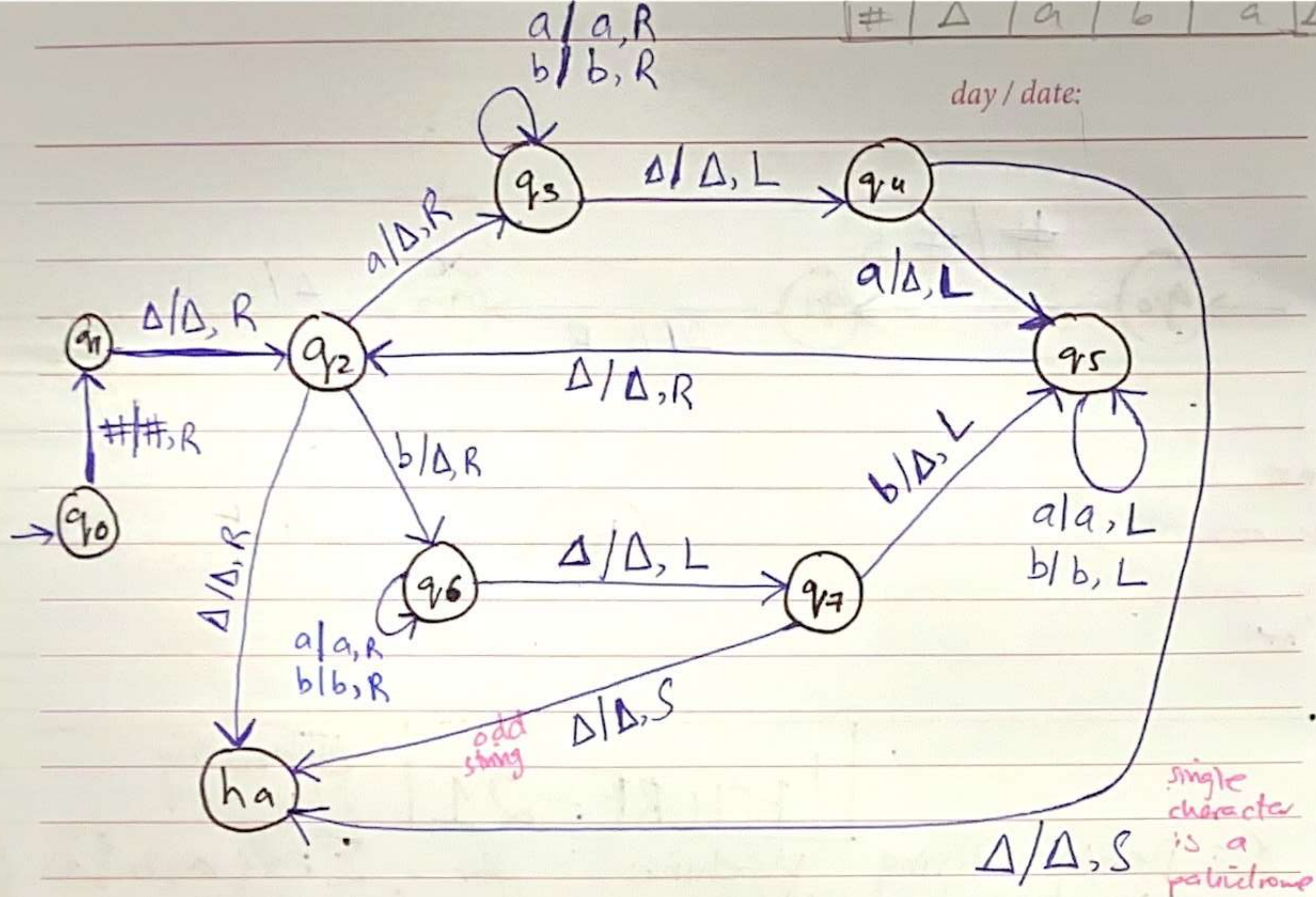
Q: Develop Turing Machine for $\Sigma = \{a, b\}$.
 $L = \{u \mid u \in \Sigma^* \text{ \& \& } u \text{ is a palindrome}\}$.

- If array's data need to be preserved; we'll keep replacing the letters with their corresponding capital letters (A or B). In this case we'll give a check of capital A or capital B.
- But in this ques. we'll just replace with Δ and need a single check.

LOGIC:



- Mark first 'a' after delta with 'Δ' and keep moving towards right till final delta is found. After that, move left once (last element of string / 2nd last of array) and compare it with first element which we just marked Δ in start. Then move towards left till delta found (2nd element). Then move towards extreme right again till Δ for 2nd last element of string to compare with 2nd element. REPEAT till we reach center from both sides.



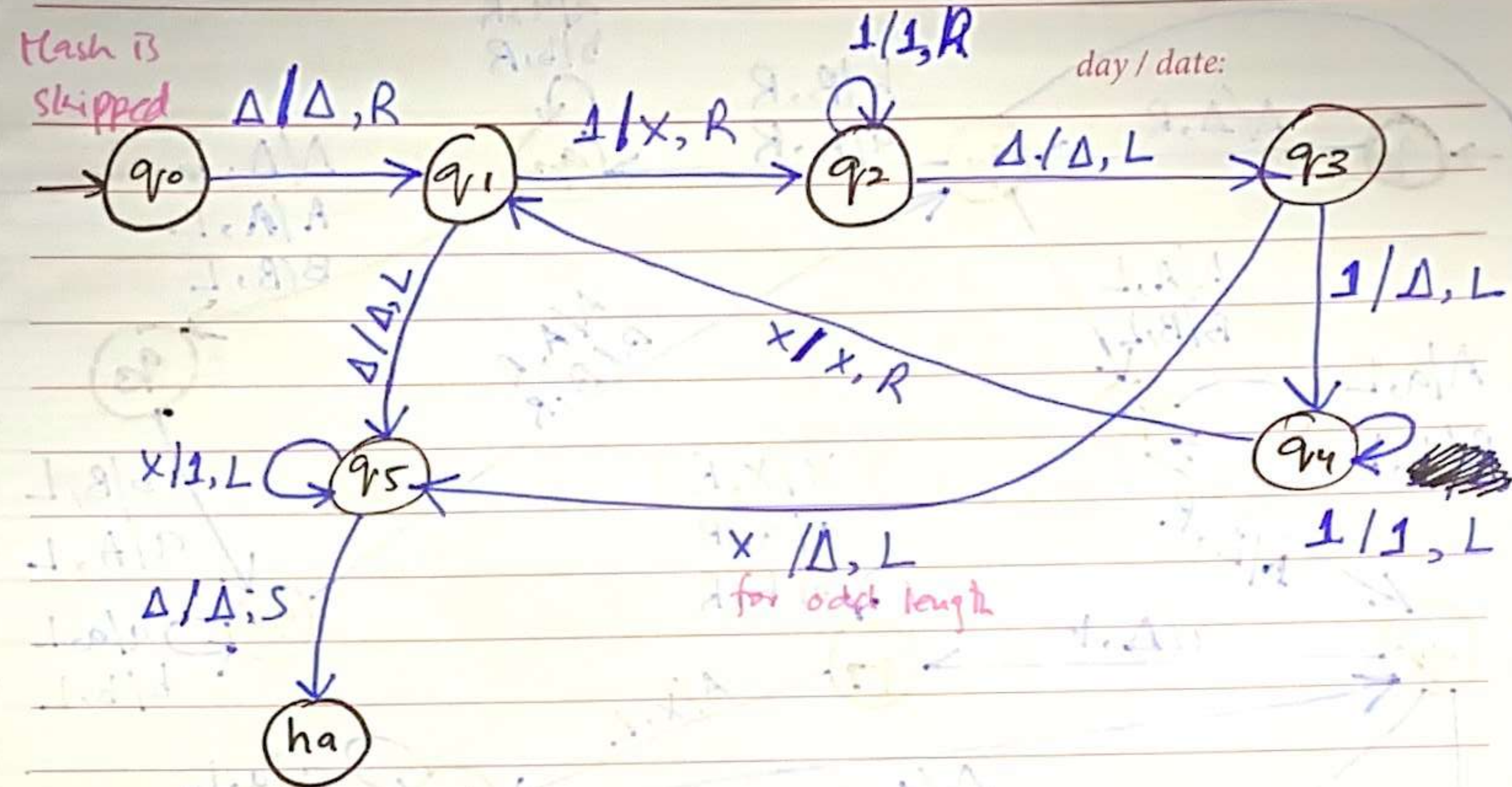
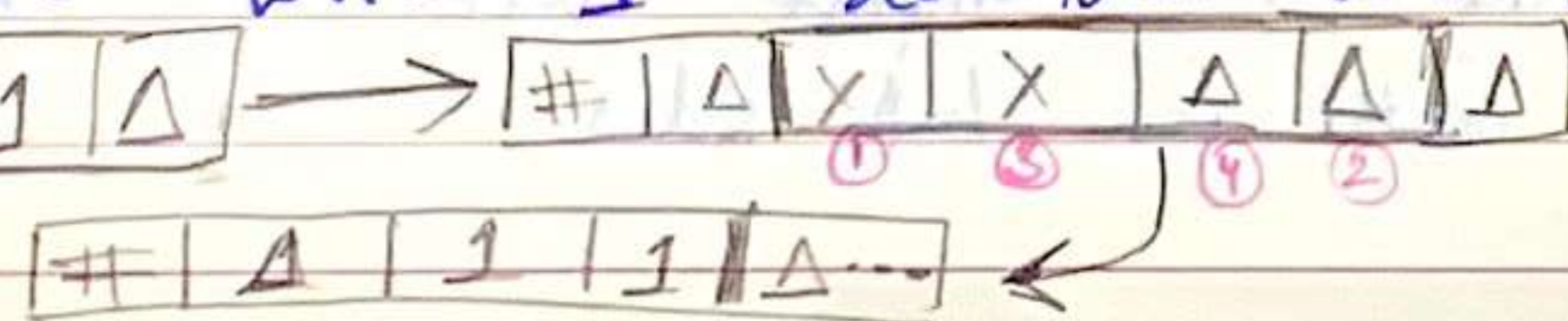
LECTURE - 22

Thursday
11/11/25

{a, b}

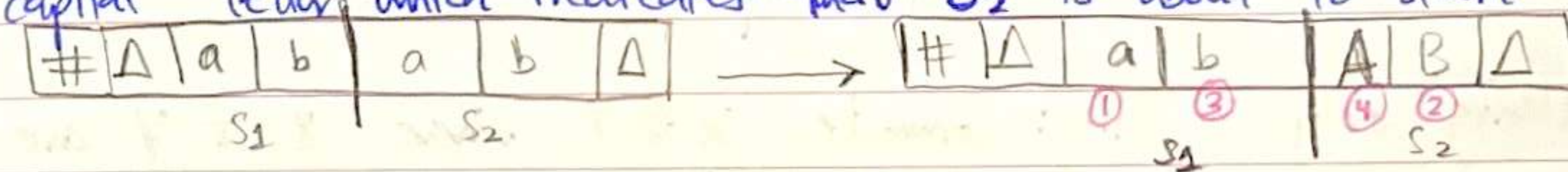
Q: (a) $L = \{x \mid x \in \Sigma^* \text{ and } n = SS \text{ where } S \in \Sigma^*\}$
 (b) $\Sigma = \{1\}$, $n \in \Sigma^*$ (unary input). Divide n by 2 and store no. of ones equal to quotient.
 Input = 11111 (means 5 in decimal value)

Logic (b): Start with MSB being replaced by 'X'. Then move towards LSB 1 and replace with 'Δ'. Then move back towards new MSB 1 (i.e. second 1 from left) and replace again with 'X'. Again move towards new LSB 1 and replace with 'Δ'. When there'll be no LSB 1 on right side of 'X', means all ones finished. ∴ we'll replace all the 'X's' with '1' as that will be quotient.

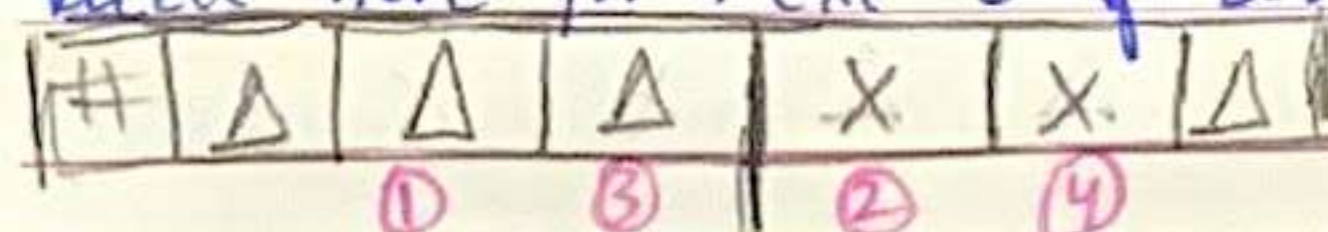


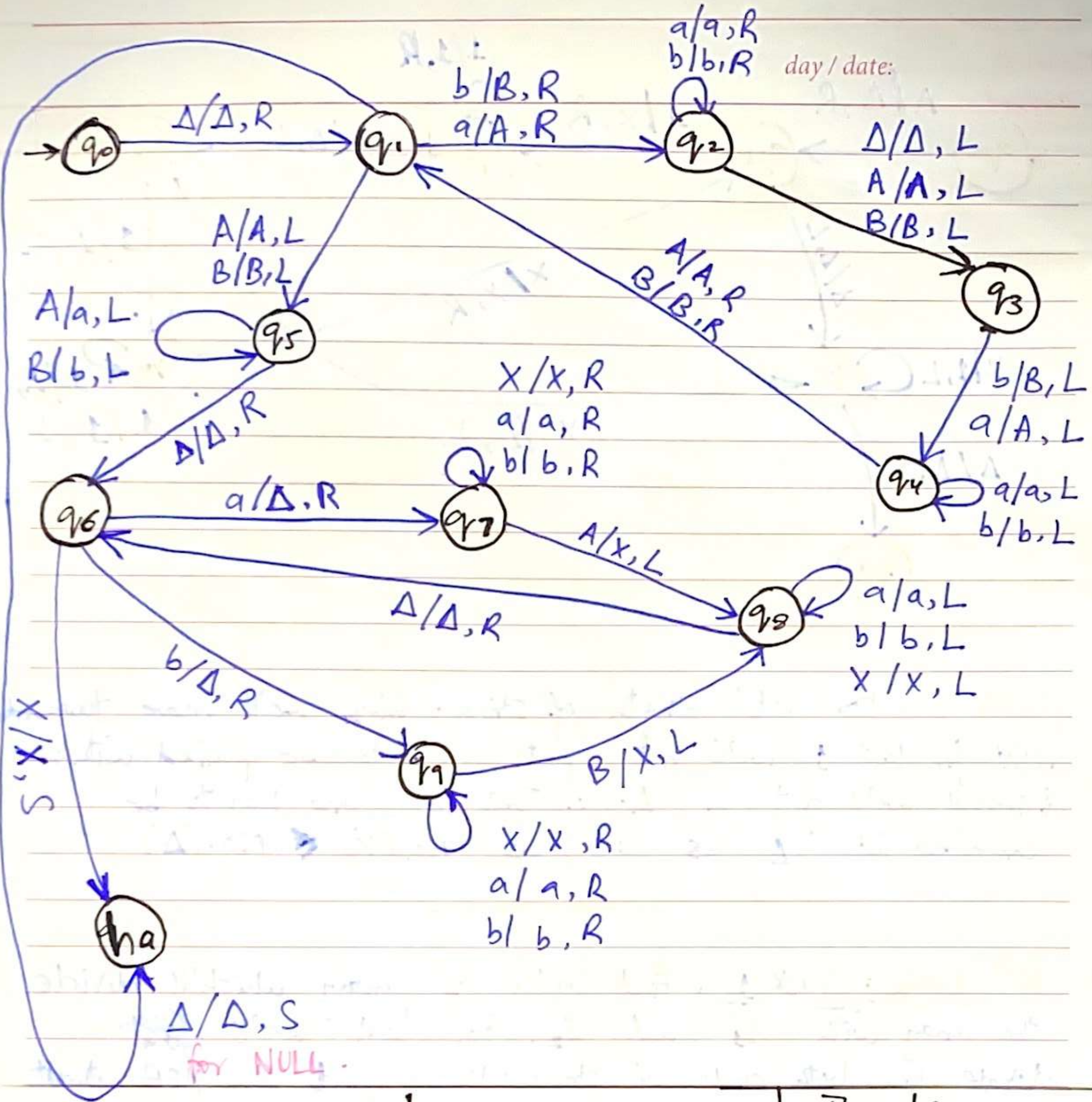
[NOTE]: For odd length of string; when we'll move towards right for last 1, we'll not get a 1 to be paired with; instead we'll get a Δ. ∴ That last one has to be converted into Δ as well, as No. (X) < No. (Δ).

(a) Logic: Task 1: find center of array which'll divide the array with S_1 and S_2 . For that we'll iterate from both ends of string. Means first from MSB without replacing anything and then from LSB, but here we'll replace with capital letters. This will be done until on the right side of the last element of S_2 (middle) is a capital letter, which indicates that S_2 is about to start.



Task 2: Now we'll start comparing from first element of S_1 with first element of S_2 . Replace S_1 elements with 'X' to come back here for next comparison. Similarly replace S_2 elements with 'X'.





LECTURE - 23

Thursday 13/11/25.

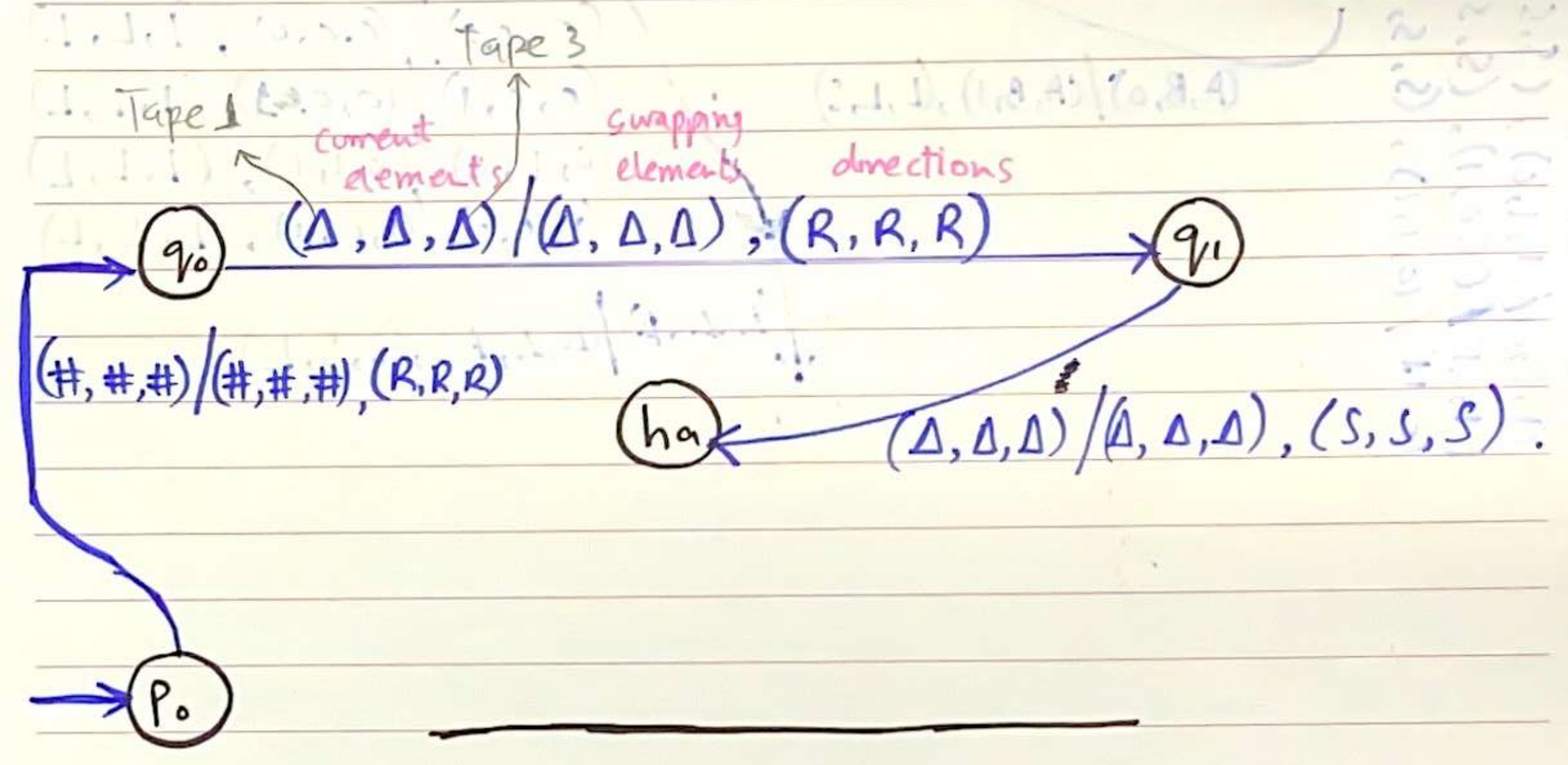
Multiple Turing Machine (2D Array): Single TM are combined with each having separate heads.

Q: Using only 3 TM; Compute $x \oplus y$ where x & y are two binary no. of equal length.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Normally Tape 1 & 2 are input & Tape 3 is output.

Tape 1	#	Δ	0	1	1	0	Δ
Tape 2	#	Δ	1	0	1	1	Δ
Tape 3	#	Δ	1	1	0	1	Δ



Q: Suppose you have 2 binary numbers x & y of equal length. Perform $x + y$ (addition). You're 3 TM.

x	y	Carry Bit	Som	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

LOGIC: Add x & y , and store the carry in TM:3, then keep on adding. The answer would be in $n+1$ digits due to carry. $\therefore$ TM:3 will have an extra no. Move towards extreme right first in all 3 TM and then start adding. [NOTE]: Head of TM:3

(i.e. answer TM) will move towards extreme right with an extra cell on right for $n+1$ answer.

$$(A, B, \Delta) / (A, B, 0), (R, R, R)$$


~~Ex~~ Cases for a regular language. that cannot be handled.

- 1) equal no. of a's & b's ($n \mid u \in \Sigma^* \& u = a^n b^n$)
- 2) nested structures

i) $\{a^n b^n c^n \mid n \geq 0\}$: multiple counting isn't possible in one stack

★ Use pumping lemma to prove it non CFL.

2) Exponential functions

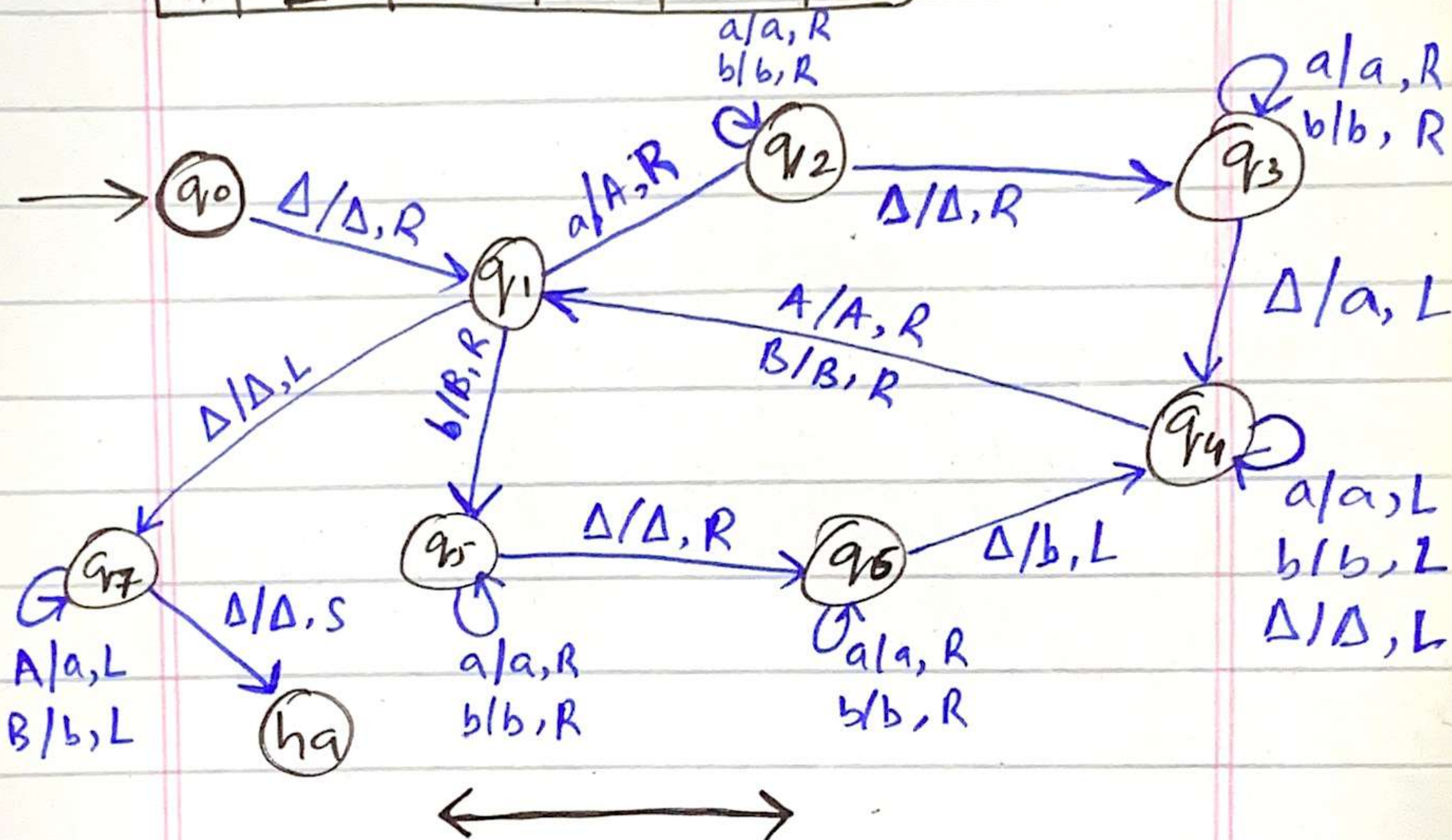
sday
8/11/25

LECTURE-24

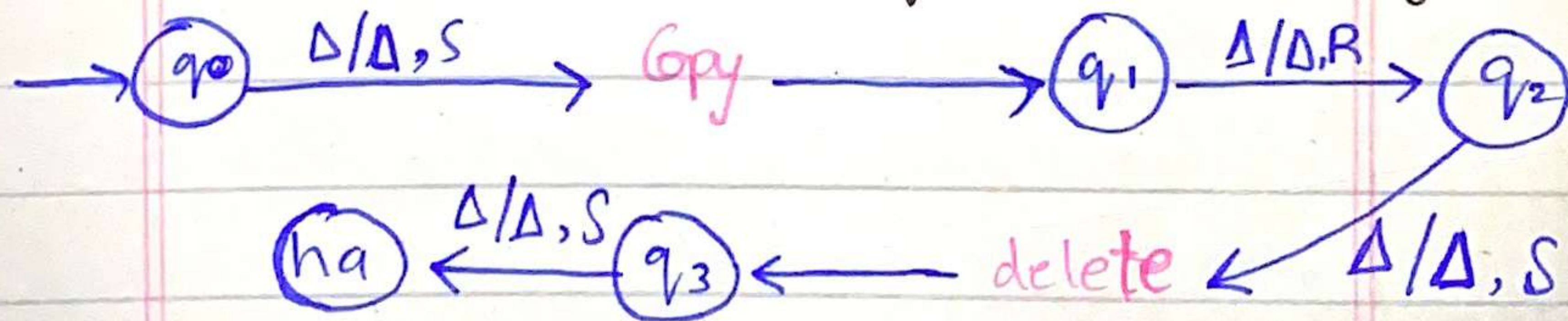
Q1: Input $\Delta x \Delta$. Copy x after 2nd Δ .
Head should be on 1st delta. $\Sigma = \{a, b\}$.

#	Δ	a	b	a	Δ
---	----------	---	---	---	----------

 $\rightarrow \Delta aba \Delta aba \Delta \dots$



$\Rightarrow$ Combining Two Turing Machines Using func.



* 'Copy' and 'delete' are two diff. TM.

25/11/25.

LECTURE-25 Tuesday



Recursively Enumerative language:

They can accept, reject, crash or infinitely loop the language.

NOTE: Whereas; simple recursive lang. either just accepts or reject the lang.

→ Example: "Halting Problem".

Checking if the entire TM tape ~~is~~ has any non-delta value. Since the right side of Δ 's are always utilizeable, means we can place the character where-ever we want or not place at all. Therefore it will loop infinitely.

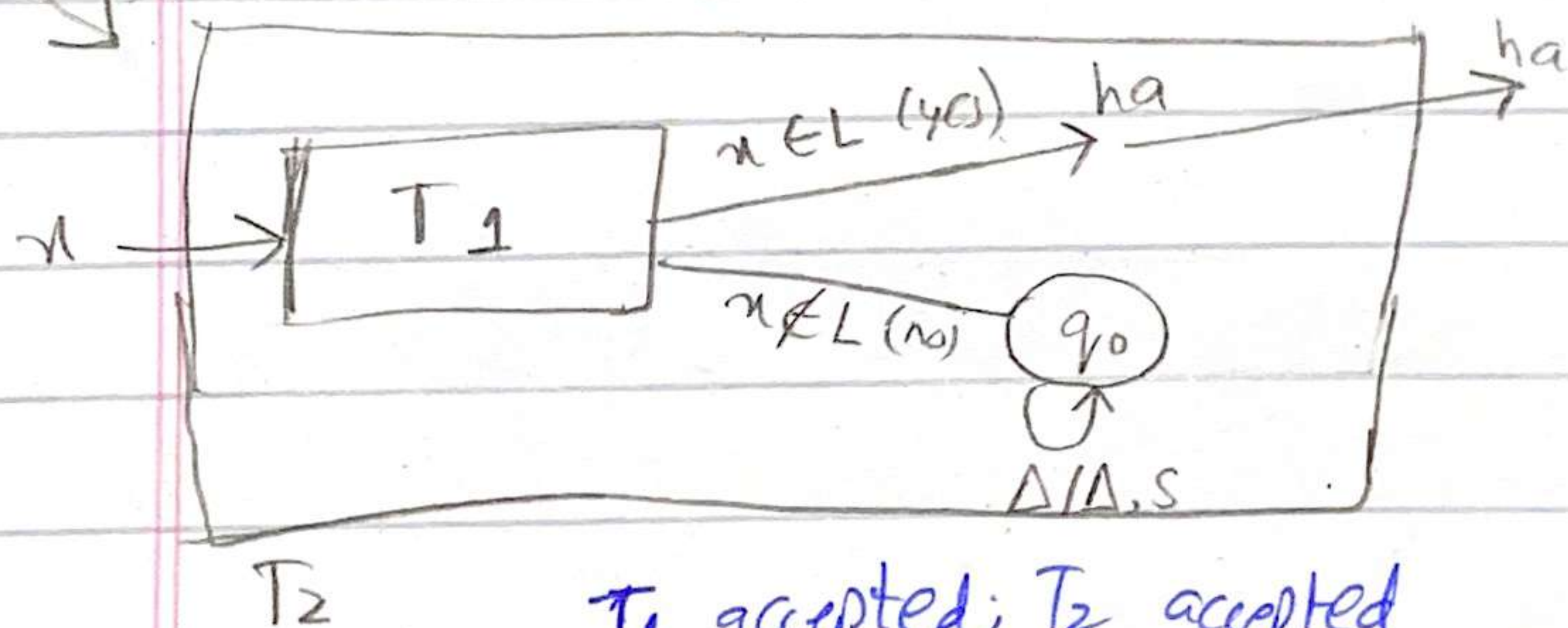
→ Every recursive lang. is recursively ^{me}enumerable recursive lang. This is because in the rejecting part of recursive lang; we can add a state after it which will loop forever.

complement: accept will reject
reject will accept.

with " $\Delta/\Delta, S$ ". Means it's looping infinitely but still gives reject.

★ Every recursively enumerable lang. isn't simple recursively lang.

⇒ Diagram: ★ T_1 : recursively lang.
 T_2 : // enumerable lang.

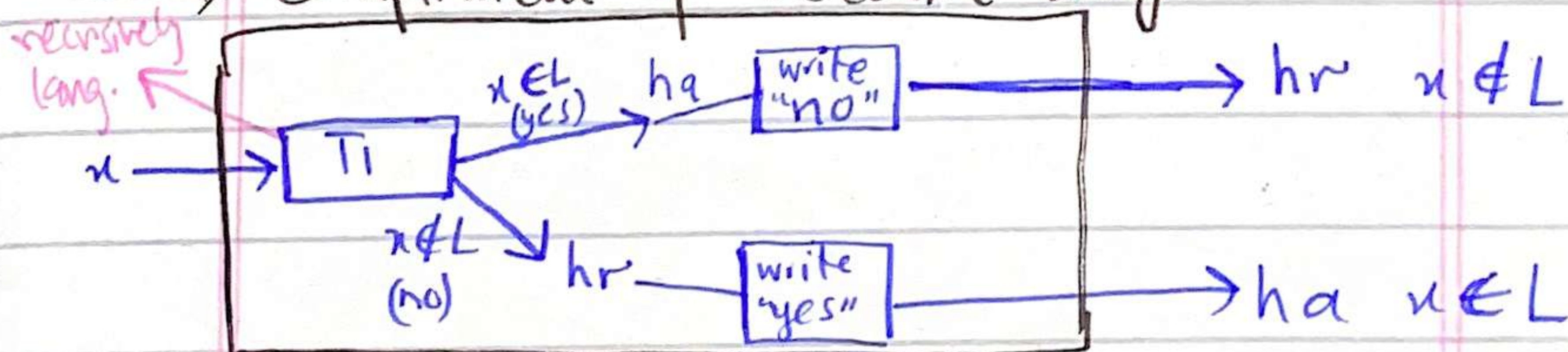


T_1 accepted; T_2 accepted

T_1 rejected; T_2 infinity loop

} recursive enumerable lang.

⇒ Complement of Recursive Lang:



At acceptance of T_1 (Recursive); T_2 (Complement) is being rejected and vice versa for rejected.

★ T_2 complement of T_1

NOTE: Compliment of Recursively Enumerable lang; is Non-Recursively Enumerable lang which does not have Turing Machine.
This is because we cannot go back from infinite ^(rejection) $\uparrow$ to acceptance.

$\Rightarrow L_1 \& L_2$ are Recursive:

- $L_1 \cup L_2$: Recursive lang.
- $L_1 \cap L_2$: Recursive lang.

$\Rightarrow L_1 \& L_2$ are Recursively Enumerable:

- $L_1 \cup L_2$: Recursively Enumerable
- $L_1 \cap L_2$: $//$ $//$

Practice Q/s:

Single Tape for:

- ① $a^i b^j c^k$: $i = j = k$
- ② $a^i b^j c^k$: $i > j + k$
- ③ $\text{No.}(a) = \text{No.}(b) = \text{No.}(c)$

↓
Ans - Find 'a'; capitalise. and then move head to start. Find 'b' and capitalise. Move back to head and then find 'c'; capitalise. While finding 'c'; ignore all 'a', 'A', 'b', 'B', 'c'.